MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF REPUBLIC OF MOLDOVA
TECHNICAL UNIVERSITY OF MOLDOVA
FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE ENGINEERING AND AUTOMATICS

CodeCraft: learning programming through Minecraft

Project report

Mentor: Crucerescu Vladislav
Students: Chiril Boboc, FAF-242
Vasile Brînză, FAF-242
Cristian Bruma, FAF-242
Gabriela Bîtca, FAF-242
Teodor Strulea, FAF-242

Chișinău, 2026

Abstract

CodeCraft is a domain-specific language (DSL) designed to enable children aged 10–15 to learn programming by scripting actions within the Minecraft game environment. The project was developed by Chiril Boboc, Vasile Brînză, Cristian Bruma, Gabriela Bîtca, and Teodor Strulea, students of Software Engineering at the Technical University of Moldova.

The motivation for this project stems from the disconnect between traditional programming education and the interests of young learners. Existing tools either rely on abstract block-based environments or demand technical knowledge beyond the reach of beginners. CodeCraft addresses this gap by providing a text-based language with minimal syntax, immediate visual feedback, and a meaningful context rooted in a game children already engage with.

The report covers a domain analysis examining the educational context, target user research, and the analysis of the solution, followed by the language design chapter, which details the DSL requirements, formal grammar, and the implementation of the lexer and parser components.

Keywords: domain-specific language, programming education, gamification, Minecraft, computational thinking.

Content

Introduction

The studied domain of this project sits at the intersection of programming language design and educational technology, specifically focused on the use of domain-specific languages as tools for teaching computational thinking to children. As digital literacy becomes an increasingly essential skill, the question of how to introduce programming concepts to young learners in an engaging and effective way has attracted significant research attention across both computer science and pedagogy.

The motivation for this project stems from a clear gap in the existing landscape of educational programming tools. Block-based environments such as Scratch successfully lower the entry barrier but create a problematic disconnect from real text-based programming, making the eventual transition to languages like Python or Java unnecessarily abrupt. On the other end of the spectrum, tools that operate within Minecraft — a game already deeply familiar to the target age group — such as ComputerCraft, impose technical complexity far beyond what a beginner can reasonably handle. CodeCraft was conceived to occupy the space between these two extremes: a text-based language simple enough for a ten-year-old, yet grounded in real programming syntax and capable of producing immediate, visible results inside a game children already care about.

The relevance of the topic is supported by a growing body of empirical research demonstrating that game-based, visually immediate programming environments significantly improve motivation and computational thinking outcomes among young learners. Minecraft in particular has seen documented adoption across primary and secondary education in over one hundred countries, yet accessible programmatic scripting tools tailored to novice children remain scarce, making this a timely and practical contribution.

The general objectives of the project are threefold: to design a domain-specific language suited to children aged 10–15 with no prior programming experience, to define its formal grammar and core language constructs, and to implement the foundational components of its toolchain, namely the lexer and parser.

The research methodology combined a structured literature review of educational technology, programming pedagogy, and DSL design with an empirical-informed design process, drawing on studies covering learning curves across programming paradigms, identifier readability, and common beginner mistakes to justify each design decision.

The report is organized into two main chapters. Chapter 1 presents the domain analysis, covering the educational context, a critical examination of existing tools, their shortcomings, the proposed solution and its use case scenarios, and a profile of the target users and stakeholders. Chapter 2 addresses the language design, detailing the DSL requirements, the formal BNF grammar, and the implementation of the lexer and parser components. The report concludes with a summary of outcomes and directions for future work.

1 Domain Analysis

1.1 Educational Context and Learning Objectives

In today's tech-driven world, there's a growing need to develop skills for navigating the digital era. Computational thinking (CT) has become vital, fostering analytical and problem-solving skills crucial for coding proficiency [1]. Integrating programming skills into education can enhance CT abilities, that are essential for future readiness, particularly among primary school students [2].

1.1.1 Educational programming tools for children

Typically, children begin learning programming at the stage of middle school. One of the tools they are taught with, is Scratch, a popular simplified block-based language. Scratch's aesthetic aspects are the most significant component for developing computational thinking [3]. Research indicates that game-based programming environments like Scratch serve as crucial factors for improving education quality by changing student attitudes toward the frustrations of learning programming logic and fostering skill development through experimentation in motivating contexts [4]. Other effective tools for teaching programming are Kodable, CodeMonkey, BlocklyGames. Their main effectiveness in developing computational thinking comes from their gamified design, which engages children by providing immediate feedback and having a memorable interface [5].

1.1.2 Gamification of learning programming

The integration of game elements into the learning process can contribute to the development of algorithmic thinking, logic and creative abilities of students from an early age [6]. Game methods of teaching programming differ from others in that the game is a natural learning environment that attracts students, increases their motivation and stimulates active research, allows students to apply theoretical knowledge in practice, receiving immediate feedback and observing the results of their work, promotes the development of not only technical programming skills, but also such important qualities as creativity, critical thinking and problem-solving skills, and is an interactive process that allows students to interact with educational material in a dynamic and exciting environment, creates a center for cooperation and exchange of experience between students, contributing to the development of team skills [7]. The perceived enjoyment and social influence yielded by gamified programming instruction have a high impact on student's motivation to learn programming [8].

1.1.3 Educational DSL Precedents and Benefits

Logo represents one of the earliest educational DSLs, designed in 1967 to teach programming concepts through turtle graphics and LISP-like constructs [9]. Game Maker Language similarly combines simplified commands derived from GPLs (General Purpose Languages) to enable users to create games

without advanced programming knowledge [9]. Well-designed DSLs improve programmer productivity, addressing key challenges in software development [10].

1.2 Problem Analysis

Modern children face severe attention span challenges that make traditional programming education ineffective. Standard programming courses require prolonged focus, abstract thinking, and patience for delayed results. Interface complexity becomes an obstacle instead of supporting the learning process. These requirements clash with how children consume information today.

Speaking of existing tools, there are technical barriers. Children face Java requirements far above beginner level when working with Minecraft mods. Tools like ComputerCraft use Lua syntax that confuses young learners. Setup complexity takes time before anyone writes a single line of code, which stops learning early. Command blocks, an in-game programming interface, overwhelm beginners from the start. Parents or teachers must handle installation and configuration because children lack the technical knowledge.

As for pedagogical gaps, children learn abstract concepts without seeing corresponding actions in the game. Block-based tools hide text syntax and create barriers when transitioning to real code. Motivation drops when results feel distant or unclear. Available tools do not match how children interact with digital products today. Educational resources assume adult knowledge or prior programming experience.

Pain Points in Existing Solutions for Teaching Programming:

- **Scratch and Other Visual Languages:** Scratch lowers the entry barrier by removing syntax errors, but it operates in an abstract environment disconnected from the games children already play. The visual block system hides real text-based programming syntax, which makes the transition to textual languages like Python or Java difficult [11]. As projects grow in complexity, block-based coding limits the types of programs learners can build [12]. Learners often struggle to connect their work to familiar gaming environments, causing engagement to drop [13].
- **Minecraft Education Edition:** Minecraft Education Edition integrates learning into a familiar game, but access requires institutional licensing, limiting home use [14]. The platform targets classroom instruction and emphasizes guided lessons over open-ended experimentation [15]. Activities rely heavily on pre-built curricula, which reduces creative freedom and limits exploration. Many children prefer the regular version of Minecraft where friends play, making the educational edition feel isolated from their social gaming environment [16].
- **ComputerCraft:** ComputerCraft introduces real programming concepts through Lua, but it requires text-based coding from the start. Installation and setup demand technical knowledge often unavailable to parents. Documentation assumes prior programming experience, which can overwhelm beginners. Young learners face high complexity before achieving results, leading to frustration and early disengagement [17].

– **Other Common Approaches and Their Limitations:** Minecraft command blocks allow in-game automation, but dense syntax produces frequent errors that confuse beginners. Redstone teaches logic indirectly, and debugging circuits is unclear, making reasoning about failures difficult. Traditional programming languages such as Python or Java require setup and configuration before learners can see results; syntax errors in early stages often block progress and reduce motivation.

Thus, the problem statement is the following: **Children disengage from programming education when learning tools fail to provide immediate visual feedback and do not connect abstract coding concepts to activities that sustain their interest.**

1.3 Solution Proposal

Young learners need a programming environment where code produces instant visual results within a game they love. Minecraft offers a solution because children already invest hours in the game and want to improve their gameplay, additionally, it also offers an interactive digital environment that closely aligns with core programming and computational thinking concepts. Actions performed by players occur in a clear sequence, which reflects the execution of instructions in imperative programming. Repetitive gameplay activities such as mining blocks, farming resources, or crafting items correspond to loop structures, where the same operation is repeated until a specific condition is satisfied. Decision-making in response to in-game situations, including reacting to nightfall, selecting appropriate tools, or avoiding hostile entities, mirrors conditional logic used in programming. Managing inventory items is similar to dealing with variables that change over time, while recurring building patterns and construction methods resemble the use of procedures and modular design. In-game events such as mob spawning or environmental changes parallel event-driven programming models. These conceptual parallels have been identified and discussed in multiple studies examining the educational use of Minecraft in STEM and computational thinking contexts [18, 19].

A study focusing on Minecraft-based coding activities with middle school students reported statistically significant improvements in computational thinking dimensions such as sequencing, loops, conditionals, events, and parallelism following a structured instructional intervention [20]. Systematic reviews of empirical studies from the past decade also find that Minecraft can help learners grasp basic programming concepts, provided that learning goals and instructional support are clearly defined [19, 18].

Minecraft has seen substantial adoption in formal educational contexts, primarily through Minecraft Education Edition, which is designed specifically for classroom use. According to Microsoft Education, tens of millions of teachers and students across more than one hundred countries have access to licensed versions of the platform, indicating widespread institutional uptake rather than isolated experimentation [14]. A systematic literature review covering studies published between 2015 and 2024 confirms that Minecraft has been implemented across primary and secondary education, most frequently within STEM subjects,

and that many educators continue to use it beyond initial trials [18]. Although precise global percentages of schools using Minecraft are not consistently reported, the scale of adoption and the volume of peer-reviewed research demonstrate that Minecraft is an established educational platform.

Research on the educational effectiveness of Minecraft consistently reports increased student engagement, motivation, and participation when the platform is integrated into structured learning activities. Studies grounded in constructivist and social constructivist learning theories highlight that Minecraft supports inquiry-based learning, problem solving, collaboration, and spatial reasoning [21, 22]. Experimental research further indicates that Minecraft-based interventions can lead to measurable improvements in spatial and cognitive skills, provided that instructional goals are explicit and teacher guidance is present [23].

At the same time, the literature identifies significant limitations related to technical and programming complexity. Multiple studies report that educators often struggle to design effective Minecraft-based lessons due to limited programming knowledge and lack of confidence with the platform's technical features [24, 19]. A recent systematic review emphasizes that insufficient teacher training and high technical barriers frequently result in Minecraft being used in a superficial manner, which reduces its potential to support deeper learning outcomes [18]. These findings indicate that Minecraft's educational benefits are strongly dependent on pedagogical structure and accessible tooling.

This gap creates an opportunity for a domain-specific language that turns programming into an engaging experience. Therefore, the proposed solution is the development of a domain specific language that is suited for children to script actions in Minecraft, with the purpose of learning programming.

1.3.1 Specific Problems the DSL Solves

- **Lack of Immediate Visual Feedback:** Children write code and expect to see action happen in the game world. Existing tools separate code execution from visible results. Learning slows when outcomes stay invisible or appear only as text in a console. The DSL runs commands directly inside the game environment. Children watch their bot perform actions the moment they execute the script. Blocks appear, items move, and structures are built in real time while the code runs.
- **Weak Path Toward Real Programming:** Children learn logic through drag-and-drop blocks, then face text-based code later without preparation. The transition feels abrupt and confusing. The DSL uses simple text commands from the beginning. Children read and write actual code syntax instead of clicking visual blocks. The syntax remains clear and readable while teaching real programming structure. When children move to other languages later, they already understand how text-based code works.
- **Excessive Technical Complexity:** Current tools require deep technical knowledge to accomplish basic tasks. Setting up mods, understanding game mechanics, and navigating complex interfaces create a steep learning curve. The DSL hides these complex mechanics behind clear, simple commands.

Children focus on programming logic and problem structure instead of fighting technical barriers. Commands abstract away implementation details while still teaching fundamental concepts.

- **Poor Alignment with Modern Learning Patterns:** Traditional programming education uses long tutorials with delayed gratification. Children lose interest before seeing meaningful results. The DSL embeds learning directly inside Minecraft, where children already spend time. Each command triggers a visible change in the game world. Engagement stays high because every action produces immediate outcomes that children care about.
- **Missing Support for Independent Learning:** Children depend on adult help to progress through existing programming tools. Exploration stops when guidance becomes unavailable. The DSL supports independent trial and error through clear syntax and fast feedback loops. Children experiment with commands, see what happens, fix mistakes when things go wrong, and advance on their own. Parents without programming knowledge no longer need to intervene constantly.
- **Lack of Meaningful Context:** Programming exercises in traditional curricula feel arbitrary and disconnected from student interests. Children solve problems they do not care about, so motivation fades quickly. The DSL ties every coding task to goals children set for themselves in Minecraft. Building a house, automating a farm, or organizing inventory all serve purposes that children choose. Each line of code accomplishes something personally meaningful.

1.3.2 Use Case Scenarios

- **Build a House:** The goal is for the user to place blocks in a consistent pattern to form a simple structure. A sequence of movement and placement commands is written to define where each block goes. Correct results depend on executing commands in the right order, and repeating similar actions helps reinforce the idea of structured sequences and repetition.
- **Create a Farming Bot:** The objective is to plant crops in a straight line efficiently. A repeat instruction combines movement and planting actions so the same steps are applied across multiple blocks. This use case introduces loops and demonstrates how automation reduces manual effort.
- **Manage Inventory:** The goal is to remove or move items from specific inventory slots. Inventory and drop commands are written with parameters such as slot numbers and coordinates. This use case teaches how structured commands and parameters control precise behavior.
- **React to Conditions:** The objective is for the character to respond to changes in game state, such as eating when hunger drops below a certain level. A condition is defined with an associated action. This use case introduces decision-making and conditional logic.
- **Chest Management Bot:** The goal is to organize collected items into storage chests by type. A bot is programmed to move through a base, identify items, and place them into the correct chests. This use case combines movement, logic, and automation, reinforcing data organization and control flow

through a practical task.

1.4 Target User Research

1.4.1 Age Ranges and Skill Levels of Target Users

The DSL's target users are early adolescents, approximately 10–15 years of age, who are mature enough to read and type but are still novices in programming. The age ranges were inspired from studying existing programs; for example, the new Irish curricula introduce coding in "Junior Cycle" (approx. 12–15 years of age) [25]. These users have usually mastered basic computational thinking (often via visual tools) and are ready to tackle syntax-based tasks. The DSL being text-based implies that the learners must have sufficient typing skills, although it assumes no prior familiarity with programming syntax, as it is designed for learning. The DSL is aimed at primary-school and middle-school aged children transitioning from block-based environments toward text, who can type steadily and are building their first independent coding skills [25, 26].

1.4.2 Accessibility Requirements and Learning Challenges

Accessible design must accommodate young learners. From a learning standpoint, novices universally face conceptual hurdles. Reviews find that beginners often lack familiarity with syntax, algorithms, and debugging strategies, as they struggle with syntactic, conceptual, and strategic knowledge in introductory programming [27]. Common errors include forgetting punctuation or misunderstanding loops. The DSL must thus minimize unnecessary complexity through clear error messages and gradual feature introduction, and build on children's math and problem-solving skills, since those are known challenges in learning to code [27, 28].

1.4.3 Influence of the DSL on Adjacent Users

The choice of a text-based DSL affects not only the target user but also those around them. Text-based code influences novices to rely on the teacher for syntax help, so instructors become more involved in technical guidance [29]. The DSL shapes classroom culture; an intuitive, playful coding environment makes the class friendly and encourages peer interaction [29]. In practice, this means that if the DSL is engaging, siblings or classmates might be more inclined to join in or learn together. Likewise, parents may take on roles as helpers or reviewers of the child's code, and their experience will be influenced by how approachable the text language is. In sum, the DSL influences educators' teaching style and peers' collaboration.

1.4.4 Stakeholders Involved

Besides the children, multiple stakeholders are involved in the solution. Key educational stakeholders include teachers, school administrators, and parents [30], since they guide and support the child's learning process. Curriculum designers and educational researchers are also stakeholders, as they determine how a new language fits into learning standards and lesson plans [31]. In practice, administrators and

policymakers decide whether programming is part of the school curriculum, directly affecting the DSL's adoption. In summary, the ecosystem includes the student coders, their families, their teachers, the school leaders, and the broader education community — curriculum experts, policymakers, and developers — all of whom play roles in the design, deployment, and success of the DSL [30, 31].

2 Language Design

2.1 DSL Requirements

2.1.1 General Language Rules

Research indicates that children consistently struggle with strict syntax requirements, punctuation rules, language formalities, verbose statements, and inadequate debugging feedback [32]. Python addresses these challenges through its minimal syntax and readable structure, which explains its widespread adoption in introductory programming courses across universities worldwide [32]. These findings informed the decision to adopt Python as the primary source of inspiration for the DSL's syntax and overall design.

Empirical research on identifier naming conventions demonstrates that different styles — camelCase, PascalCase, and snake_case — impose measurably different levels of visual effort and affect the speed at which identifiers are recognised by the reader. Studies using eye-tracking methodology found that snake_case yields the lowest visual effort and the highest recognition speed, particularly among novice programmers [33]. Consequently, snake_case was selected as the identifier style for all predefined variables and functions in the DSL.

2.1.2 Language Paradigm

Given that the DSL targets children with minimal programming experience, an imperative paradigm was selected as the foundational approach. Empirical research demonstrates that imperative programming exhibits one of the lowest learning curves among major paradigms [34]. This paradigm is particularly suitable for novice learners, as it aligns with sequential, step-by-step reasoning patterns and facilitates code comprehension during debugging processes.

While object-oriented programming presents comparable learning difficulty to imperative programming [34], it was deemed less appropriate for this context. The object-oriented paradigm emphasizes abstraction and code encapsulation, concepts that may prove challenging for the target demographic. Additionally, object-oriented approaches typically require more verbose code structures, introducing unnecessary complexity beyond the DSL's intended scope.

2.1.3 Core Language Constructs

The DSL should include variables to familiarize children with a core component of most programming languages. Variables are essential for storing coordinates, object IDs, and other Minecraft elements that users need to manipulate.

To enable process automation, the DSL will incorporate conditional statements and iterative constructs for handling repetitive tasks. As these represent fundamental components of most programming languages, their inclusion supports progressive learning objectives.

2.1.4 Abstraction Level and Complexity of Functionalities

To preserve user flexibility and support progressive skill development, the DSL will adopt a layered abstraction approach, providing both primitive and composite operations. Primitive operations include atomic actions such as block placement, directional orientation, jumping, clicking, and inventory management. Composite operations encompass higher-level tasks including wall construction, water removal from defined areas, and automated crop cultivation.

This architectural design enables users to construct custom high-level functionalities by composing primitive operations when predefined composite operations do not satisfy specific requirements. Such an approach promotes understanding of functional composition in computational thinking.

2.1.5 DSL Limitations

To prevent implementation complexity and breaking Minecraft official policies, the DSL will not change the game's state with means external to the user's in-game character. Thus, the DSL allows the user to interact with the game only through their character.

Although the usage of the DSL is intended for children, the installation process of the DSL requires external assistance.

2.2 The Grammar

The DSL follows an imperative, statement-oriented structure where a program consists of a sequential list of statements. Each statement occupies its own line, with newlines acting as statement terminators rather than semicolons, reducing syntactic noise for novice learners.

The language supports six categories of statements: variable assignments, two forms of iteration, a while loop, a conditional, function calls, and single-line comments. Comments are introduced with a double dash (`--`), a deliberate choice to avoid conflict with the subtraction operator and to remain readable.

Iteration is provided in two forms. A count-based loop (`for N times`) allows repetition without requiring the learner to manage an explicit iterator, which lowers the cognitive barrier for simple repetitive tasks. A range-based loop (`for x from A to B`) introduces the concept of an iterator variable progressively, once the learner is ready for it. Both forms use indentation-based block delimitation, consistent with Python-like conventions that children are likely to encounter in subsequent languages.

Expressions follow a standard left-recursive arithmetic structure supporting addition, subtraction, multiplication, and division, with parentheses available for grouping. A special dot-notation construct (`object.property`) is provided to represent Minecraft-specific entities and their attributes, offering a lightweight form of structured data access without introducing full object-oriented semantics.

Identifiers follow conventional alphanumeric rules with underscore support. Literals include integers, quoted strings, and boolean values (`true`/`false`). Whitespace handling is made explicit in the

grammar, with optional spacing permitted around operators and delimiters to give learners flexibility in formatting their code.

2.2.1 Grammar Notation Conventions

The grammar is written in Backus–Naur Form (BNF), where non-terminal symbols are enclosed in angle brackets and terminal symbols appear as quoted string literals. Several special tokens and conventions are used throughout the grammar and are defined as follows:

- **NEWLINE:** Represents a line break character (`\n`), used to terminate statements and signal the end of a logical line.
- **INDENT:** Represents an increase in indentation level at the beginning of a line. The grammar does not prescribe a fixed indentation width; any consistent combination of spaces or tabs is accepted, consistent with learner-friendly formatting conventions.
- **DEDENT:** Represents a decrease in indentation level, signaling the end of an indented block. It is the logical counterpart of **INDENT**.
- ε**:** Denotes the empty string, used in productions where a symbol may derive nothing, such as optional whitespace or an empty character sequence.

The complete formal grammar in BNF notation is provided in Appendix A.

3 Language implementation

3.1 Lexer Implementation

The lexer is responsible for transforming raw source text into a flat sequence of tokens, which the parser then consumes. It operates line by line, handling both the structural indentation logic and the inline tokenization of each line's content.

3.1.1 Token Recognition

Token recognition is driven by a list of ordered token specifications, each pairing a `TokenType` with a regular expression pattern. The order is significant — more specific or longer patterns are listed before shorter ones to prevent incorrect partial matches. For example, the two-character operators `!=`, `==`, `<=`, and `>=` are specified before their single-character counterparts `<` and `>`.

All individual patterns are combined at class initialization into a single master regular expression using alternation, and compiled once into a `Pattern` object. During tokenization, a `Matcher` is advanced through the content of each line using `lookingAt()`, which attempts to match at the current position without requiring a full-string match. The capturing group that produced the match is then used to identify the corresponding `TokenSpec`.

3.1.2 Indentation Handling

Since the DSL uses indentation-based block delimitation, the lexer is responsible for emitting `INDENT` and `DEDENT` tokens explicitly. An integer stack tracks the indentation levels encountered so far, initialized with a base level of zero. At the start of each line, the number of leading spaces and tabs is counted. If the measured indentation exceeds the top of the stack, an `INDENT` token is emitted and the new level is pushed. If it is less, `DEDENT` tokens are emitted and levels are popped until the stack matches the current indentation. Blank and whitespace-only lines are skipped entirely and carry no structural meaning.

3.1.3 Keyword Resolution and Special Cases

Identifiers are first matched by a general alphanumeric pattern, then passed through a keyword lookup to determine whether the matched text corresponds to a reserved word such as `for`, `while`, `if`, or `true`. If a keyword match is found, the corresponding keyword token type is emitted instead of a generic `IDENT` token.

String literals have their surrounding quotation marks stripped during tokenization, so the parser receives only the inner content. Comments, introduced by a double dash (`--`), cause the lexer to discard the remainder of the line immediately upon detection. Any character that does not match any known pattern results in an `ILLEGAL` token carrying the unrecognized character, allowing the parser to produce a meaningful error message rather than failing silently.

Each line concludes with an explicit NEWLINE token. At the end of the source, any remaining open indentation levels are closed with corresponding DEDENT tokens, followed by a final EOF token.

3.2 Parser Implementation

The parser consumes the flat token sequence produced by the lexer and constructs an Abstract Syntax Tree (AST), which represents the hierarchical structure of the program. It is implemented as a hand-written recursive descent parser, where each grammar rule corresponds directly to a parsing method.

3.2.1 Token Navigation

The parser maintains a cursor over the token list and exposes three core navigation primitives. The `advance()` method moves the cursor forward and returns the new current token. The `check()` method tests whether the current token matches a given type without consuming it. The `expect()` method asserts that the current token has a specific type, consumes it, and throws a `ParseException` with a descriptive message including the line and column number if the assertion fails. A `peek()` method allows looking ahead by an arbitrary offset, which is used for disambiguation without side effects.

3.2.2 Statement Parsing and Disambiguation

The top-level entry point, `parseProgram()`, skips any leading newlines and delegates to `parseStatement` which iterates until an EOF or, when parsing inside a block, a DEDENT token is encountered.

Individual statements are dispatched in `parseStatement()` based on the type of the current token. Control flow keywords (`for`, `while`, `if`) are routed directly to their respective parsing methods. When the current token is an identifier, a one-token lookahead is used to disambiguate between an assignment, where the next token is `=`, and a function call statement, where the next token is `(`. Any identifier-headed expression that resolves to something other than a function call is rejected with a parse error, enforcing the grammar's restriction that bare expressions are not valid statements.

3.2.3 Control Flow Constructs

The two forms of `for` loop are disambiguated after consuming the `for` keyword by checking whether the next token is an identifier followed by the `from` keyword. If so, a range loop is parsed; otherwise, a count loop is assumed. Both forms conclude by parsing an indented block via `parseBlock()`, which expects an INDENT token, recursively parses a statement list, and then expects a DEDENT token. The `if` statement optionally parses an `else` branch by checking for the `else` keyword after the then-block, with intervening newlines skipped.

3.2.4 Expression and Condition Parsing

Expressions are parsed using a standard two-level recursive descent that naturally encodes operator precedence. The `parseExpression()` method handles additive operators by parsing a `parseTerm()` and then consuming any sequence of `+` or `-` operators followed by additional terms, constructing left-associative `BinaryOp` nodes. The `parseTerm()` method handles multiplicative operators in the same manner, and

`parseFactor()` handles the base cases: numeric, string, and boolean literals; parenthesised expressions; unary negation; and identifier-headed constructs.

When an identifier is encountered in `parseFactor()`, a further one-token lookahead determines whether it begins a function call (`IDENT` followed by `()`), a dot-notation special object access (`IDENT` followed by `.`), or a plain variable reference.

Conditions are parsed separately from expressions. The `parseCondition()` method first checks for a `not` keyword, in which case it recursively parses a negated condition. Otherwise, it parses a left-hand expression and checks whether the current token is a comparison operator. If so, a `ComparisonCondition` node is produced; if not, the expression itself is wrapped in a `BooleanCondition`, supporting the use of bare identifiers and function calls as boolean predicates.

3.2.5 Error Handling

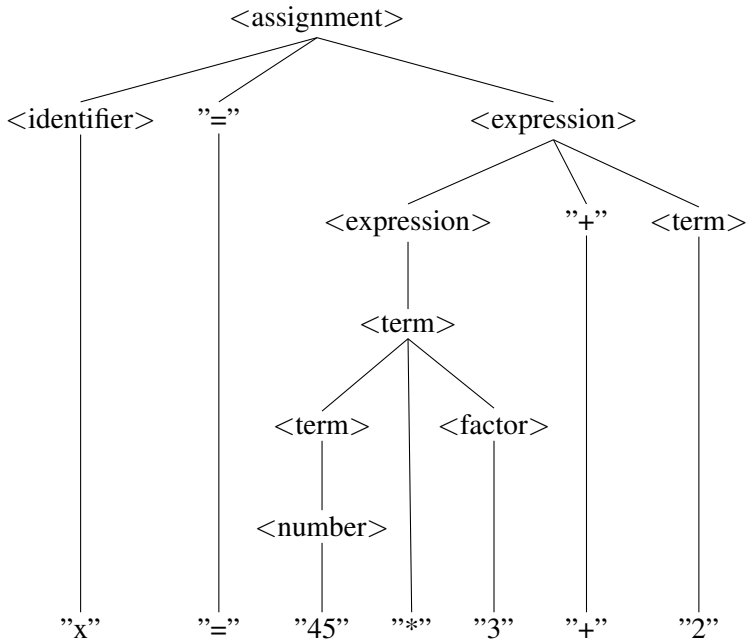
All syntax errors are reported by throwing a `ParseException`, an unchecked exception that carries a human-readable message including the offending token type, its literal value where applicable, and its line and column position. This ensures that error messages are actionable for the learner rather than presenting raw internal state.

3.3 AST Examples

The following examples illustrate the simplified abstract syntax trees produced by the parser for various language constructs. Each tree omits certain lexical details (such as spaces and optional formatting) to highlight the essential structure.

3.3.1 Variable Assignment

The assignment statement `x = 45 * 3 + 2` demonstrates correct operator precedence and nesting of expressions.

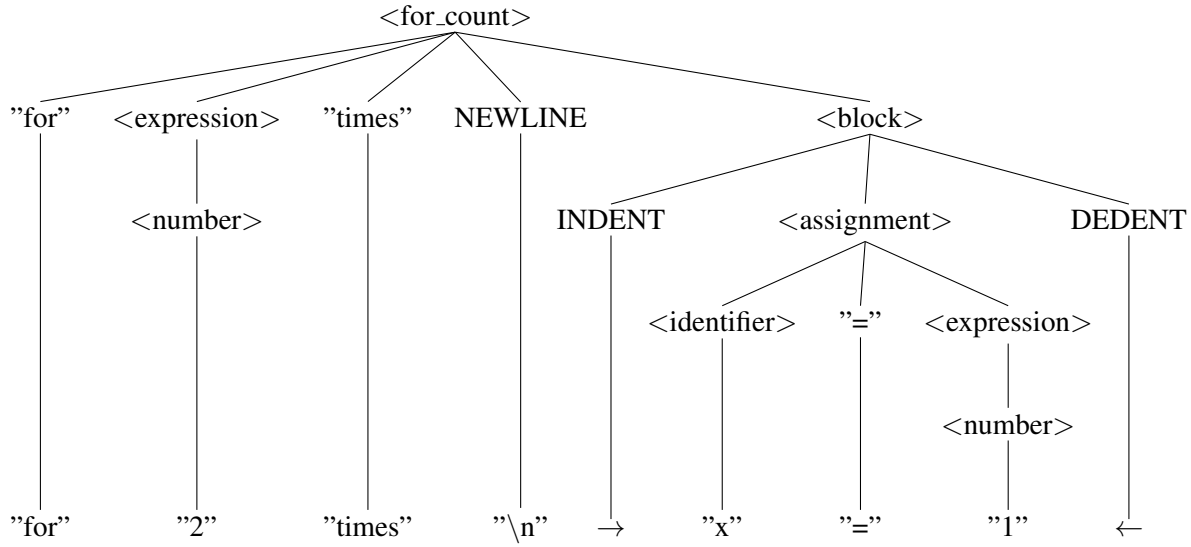


Corresponding source line:

```
x = 45 * 3 + 2
```

3.3.2 For Count Loop

The count loop `for 2 times x = 1` shows how the block structure is captured via `INDENT` and `DEDENT` markers, similar to Python.

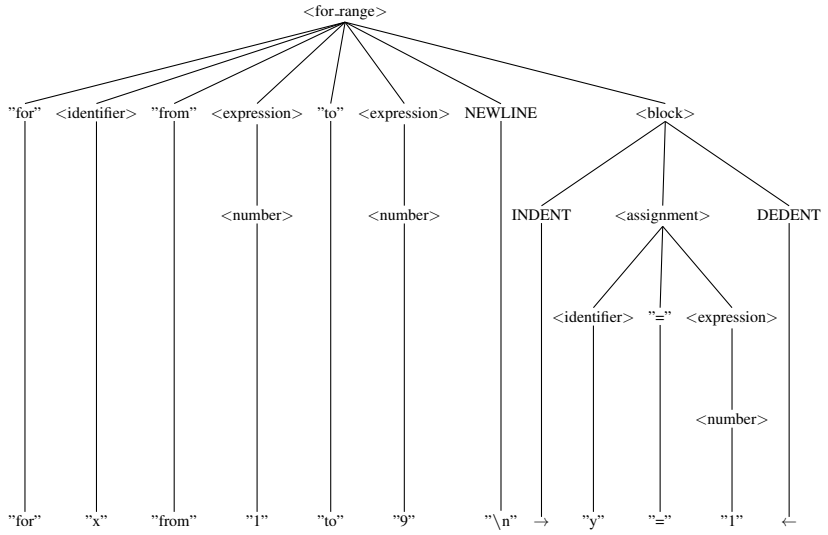


Corresponding source lines:

```
for 2 times
    x = 1
```

3.3.3 For Range Loop

The range loop `for x from 1 to 9 y = 1` iterates a variable over a sequence of values. The AST explicitly represents the loop variable, the bounds, and the body.

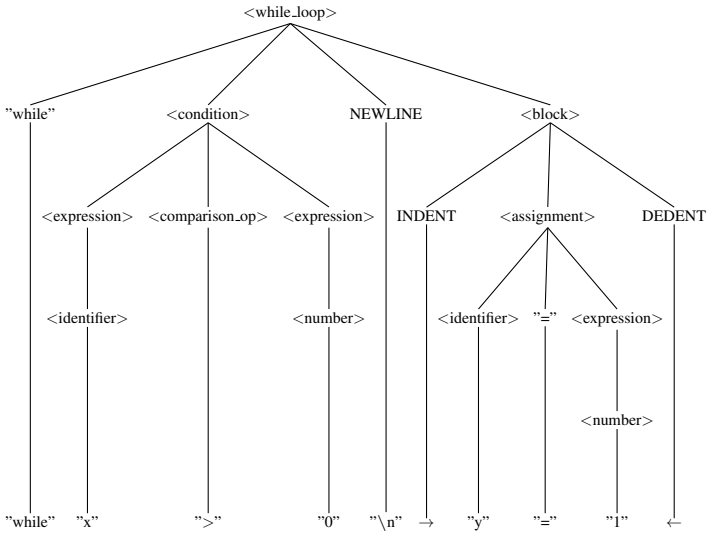


Corresponding source lines:

```
for x from 1 to 9
    y = 1
```

3.3.4 While Loop

The while loop `while x > 0 y = 1` demonstrates a condition node containing a comparison operator.

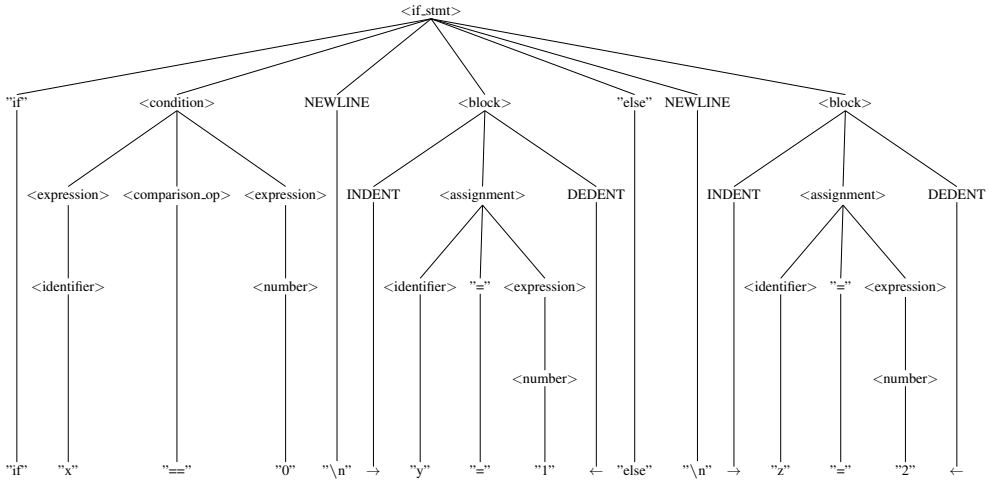


Corresponding source lines:

```
while x > 0
    y = 1
```

3.3.5 If-Else Statement

The conditional statement `if x == 0 y = 1 else z = 2` shows the structure of an if-else with two separate blocks.



Corresponding source lines:

```
if x == 0
    y = 1
else
    z = 2
```

These examples confirm that the parser correctly handles operator precedence, block structure, and

the various control flow constructs defined in the language. The simplified ASTs omit whitespace and delimiters, focusing on the semantic relationships that will be used in later compilation phases. The full variants of these ASTs are included in the Appendix E.

3.4 Interpreter Implementation

The interpreter walks the AST produced by the parser and executes the program, producing a sequence of `Action` objects that are later dispatched to the Minecraft game engine. The interpreter is intentionally data-driven, separating core AST evaluation logic from extensible function bindings and special object resolution.

3.4.1 Overall Architecture

The `Interpreter` class serves as the main entry point for execution. It maintains references to a `FunctionRegistry` (for user-defined and standard library functions) and a `SpecialObjectResolver` (for dot-notation object access), making the system extensible without coupling the traversal logic to specific function implementations. Upon initialization, an `Environment` is created to hold the program's variables, and lists are allocated for emitted `Action` objects and logged messages.

The primary method, `execute(ASTNode.Program program)`, accepts a parsed AST and returns an `InterpreterResult` containing the final value, all emitted actions, a snapshot of the final environment state, all logged messages, and a stopped flag indicating whether a `stop` statement was encountered.

3.4.2 Environment and Value Management

The `Environment` class manages variable storage using a stack of scopes. Each scope is a `LinkedHashMap Value>` mapping variable names to their runtime values. Although the current language uses only global scope, the stack structure permits future extensions for nested scopes (e.g., function-local variables) without API changes.

Variable assignment is handled by the `assign(name, value)` method, which searches the scope stack from the topmost scope downward; if the variable already exists in any scope, its value is updated in place. Otherwise, the variable is created in the topmost (most local) scope. Variable lookup via `get(name)` searches the stack from top to bottom and throws an `InterpreterException` if the variable is undefined.

The `Value` class is a wrapper around arbitrary `Object` instances, providing type-aware conversion methods. A single `NULL` instance is cached and reused for null values. The `asLong()`, `asBoolean()`, and `asString()` methods perform implicit type conversions—for example, boolean `true` converts to `1L`, and any non-empty string converts to `true`—allowing programs to mix types flexibly.

3.4.3 Statement Execution

Statements are executed by the `executeStatement()` method, which dispatches on the AST node type and handles all control flow constructs.

Assignments and Variables

An `Assignment` node is handled by evaluating the right-hand expression and storing the result in the environment under the target variable name. Assignment Execution (Pseudocode):

```
if (node instanceof ASTNode.Assignment assignment) {
    Value value = evaluateExpression(assignment.value, ...);
    environment.assign(assignment.name, value);
    return value;
}
```

For Loops

The interpreter handles two forms of `for` loops. For count loops (`for N times`), the count expression is evaluated to a long, and the loop body is executed that many times. For range loops (`for X from A to B`), the bounds are evaluated, and the loop variable is assigned to each value from `A` (inclusive) to `B` (exclusive) in sequence, allowing the body to access the current iteration value.

Both loop forms catch `StopSignal` exceptions (thrown by a `stop` statement) to enable early exit. When caught, the loop terminates immediately and execution resumes after the loop. For Count Loop Execution (Pseudocode):

```
if (node instanceof ASTNode.ForCount forCount) {
    long count = evaluateExpression(forCount.count, ...).asLong();
    Value last = Value.nullValue();
    for (long i = 0; i < count; i++) {
        try {
            last = executeStatements(forCount.body.statements, ...);
        } catch (StopSignal stopSignal) {
            break;
        }
    }
    return last;
}
```

While Loops

While loops evaluate their condition repeatedly using `evaluateCondition()`, which can handle various condition types (comparison conditions, boolean conditions, and negations). The loop body is executed as long as the condition evaluates to a truthy value. As with `for` loops, a `stop` statement breaks out of the loop.

If-Else Statements

The condition is evaluated once, and if it is truthy, the then-block is executed. If the condition is falsy and an else-block is present, the else-block is executed instead. Only one branch is ever executed.

Stop Statements

When a `stop` statement is encountered, the interpreter throws an internal `StopSignal` exception. This signal propagates up through nested loops and blocks, causing immediate termination. The top-level `execute()` method catches this signal and sets the stopped flag in the returned result.

3.4.4 Expression Evaluation

Expressions are evaluated by the `evaluateExpression()` method, which similarly dispatches on node type.

Literals

Numeric, string, and boolean literals are wrapped in `Value` objects. Identifiers are looked up in the environment.

Arithmetic and String Operations

Binary operators are handled by evaluating both operands and applying the operator:

- Addition (+) performs numeric addition if both operands are numbers, otherwise concatenates them as strings.
- Subtraction, multiplication, division, and modulo perform numeric operations after converting operands to long.
- Division and modulo by zero throw an `InterpreterException`.

Function Calls

When a `FunctionCall` node is encountered, all arguments are first evaluated to `Value` objects. An `ExecutionContext` is constructed containing references to the interpreter, environment, Minecraft context (if available), the list of emitted actions, the list of logs, and the evaluated arguments. The function is then looked up in the `FunctionRegistry` and invoked with the context. Function Call Execution (Pseudocode):

```
if (expression instanceof ASTNode.FunctionCall functionCall) {
    List<Value> arguments = new ArrayList<>();
    for (ASTNode argument : functionCall.arguments) {
        arguments.add(evaluateExpression(argument, ...));
    }
    ExecutionContext context = baseContext(..., arguments);
    return functionRegistry.invoke(functionCall.name, context);
}
```

Special Object Access

Dot notation (e.g., `player.direction`) is handled by passing the object name and field name to the `SpecialObjectResolver`, which returns the corresponding value. This design allows the standard library to register special objects without modifying the interpreter core.

3.4.5 Condition Evaluation

Conditions are parsed into dedicated AST nodes and evaluated by the `evaluateCondition()` method, which distinguishes several cases:

- **Comparison Conditions:** Both operands are evaluated as expressions, then compared using the specified operator. The result is wrapped in a `Value`.
- **Boolean Conditions:** A bare expression (e.g., an identifier or function call) is evaluated, and its truthiness is returned.
- **Negation:** A `not` keyword causes the operand condition to be recursively evaluated and negated.
- **Logical Operators:** `and` and `or` operators employ short-circuit evaluation—`and` returns false as soon as the left operand is false, and `or` returns true as soon as the left operand is true.

3.4.6 Function Registry and Function Binding

The `FunctionRegistry` maintains a map of function names (normalized to strings) to `InterpreterFunct` instances. Functions are registered by name during interpreter construction, typically via the `StandardLibrary` class, which installs all built-in functions.

An `InterpreterFunction` is a functional interface with a single method `invoke(ExecutionContext context)` that returns a `Value`. This design permits functions to:

- Inspect and modify the environment.
- Query the current Minecraft state via `minecraftContext`.
- Emit `Action` objects to be executed in sequence.
- Log messages.
- Access their arguments via the `ExecutionContext`.

The standard library includes functions for basic I/O (e.g., `log`, `print`), player movement (e.g., `move_forward`), inventory management (e.g., `open_inventory`), and timing (e.g., `wait`).

3.4.7 Error Handling

All runtime errors are reported via `InterpreterException`, an unchecked exception that carries a descriptive message. Type conversion errors, undefined variables, division by zero, and unknown functions all throw this exception with appropriate context. This approach ensures that parse-time errors are distinct from runtime errors, aiding debugging.

3.4.8 Integration with Minecraft

The interpreter is decoupled from Minecraft specifics by design. When no `MinecraftContext` is provided, the interpreter can still evaluate expressions and execute control flow; it simply cannot emit actions that require game state. The standard library's action-emitting functions (such as `move_forward`) access the Minecraft context only when needed, allowing the interpreter to remain testable in isolation.

3.5 Minecraft Integration and Action Execution

The CodeCraft DSL is executed within a Minecraft client environment where game logic proceeds at a fixed rate of 20 game ticks per second (50 milliseconds per tick). The DSL interpreter generates a sequence of discrete `Action` objects that must be executed within this real-time game loop. Many user-facing operations—such as player movement, item manipulation, and inventory navigation—require multiple ticks to complete, necessitating a sophisticated action execution framework that bridges the gap between imperative DSL semantics and Minecraft's tick-driven engine.

3.5.1 Tick-based Action Execution Model

The action execution system is built around a queue-based architecture that processes actions one at a time, with each action given an opportunity to update its state on every game tick.

The core component is the `ActionRunner`, which is registered to receive callbacks from Minecraft's client tick events. On each tick, the `ActionRunner` maintains a queue of pending `Action` objects and a reference to the currently active action (if any). The tick method follows this pattern:

```
public void tick() {
    if (current == null) {
        current = queue.poll();
        if (current == null) return;
    }

    switch (current.tick(ctx)) {
        case DONE    -> current = null;
        case RUNNING -> {}
    }
}
```

Each `Action` implements a `tick(MinecraftContext ctx)` method that returns an `ActionStatus`: either `RUNNING` (indicating that the action is incomplete and will continue on the next tick) or `DONE` (indicating that the action has completed and the next queued action should begin).

3.5.2 Multi-tick Actions and Stateful Execution

Many operations require coordination across multiple game ticks. For example, moving the player forward by a specified distance requires sustained input over many frames, whereas waiting for a fixed duration requires tracking elapsed time across ticks.

Waiting and Delay Actions

The `WaitTicksAction` is one of the simplest multi-tick actions: it simply counts elapsed ticks until the specified duration is reached. On each tick, it increments an elapsed counter and returns `RUNNING` until the counter reaches the target. This enables DSL constructs such as `wait(60)` to insert precise delays between other actions:

```
public class WaitTicksAction implements Action {
    private final int ticks;
    private int elapsed = 0;

    @Override
    public ActionStatus tick(MinecraftContext ctx) {
        return ++elapsed >= ticks ? ActionStatus.DONE : ActionStatus.
            RUNNING;
    }
}
```

Regarding player movement actions, the `MoveForwardAction` demonstrates the complexity of bridging user-level commands with real-time game physics. When a DSL program calls `move_forward(5)` to move forward five blocks, the action must:

- Compute a target position based on the player's current facing direction.
- Check that the path from the player's current position to the target is walkable (accounting for terrain, obstacles, and step heights).
- Maintain simulated input (by forcing the forward key via `InputOverride`) across multiple ticks, allowing Minecraft's physics engine to move the player naturally.
- Monitor the player's actual position each tick to ensure progress is being made toward the target.
- Detect and handle obstacles or stuck conditions, terminating early if the player cannot progress.
- Release the forced input once the player has arrived at the target, returning `DONE`.

A simplified view of the state machine is:

```
public ActionStatus tick(MinecraftContext ctx) {
    if (!initialized) {
```

```
            initialized = true;
            initializeTarget(ctx);
            if (!isPathClear(...)) return ActionStatus.DONE;  // Can't
                proceed
            InputOverride.INSTANCE.force(InputOverride.Key.FORWARD);
            forcedForward = true;
        }

        double distanceSq = distance(player, target);

        if (distanceSq <= ARRIVAL_DISTANCE_SQ) {
            InputOverride.INSTANCE.release(InputOverride.Key.FORWARD);
            return ActionStatus.DONE;
        }

        if (detectStuck(...)) {
            return ActionStatus.DONE;  // Fail-safe
        }

        return ActionStatus.RUNNING;
    }
```

Crucially, the action does not directly move the player; instead, it manipulates Minecraft's input system (via `InputOverride`) and allows the game engine to apply physics and collisions naturally. This ensures that movement respects the game's terrain, gravity, and collision detection without reimplementing Minecraft's movement logic.

3.5.3 Input Override and Single-Tick Actions

Many DSL operations can be completed in a single tick, such as selecting a hotbar slot, opening the inventory, or turning to face a cardinal direction. However, these single-tick actions must not conflict with persistent input states from long-running actions like movement.

The `InputOverride` class manages a set of forced input keys (forward, backward, left, right, jump, sneak) using an `EnumSet`. When an action forces a key (e.g., `InputOverride.INSTANCE.force(InputOverrid`) a mixin in the client input system checks this set before processing the player's actual keyboard input. If a key is forced, the forced state takes precedence; otherwise, the player's actual input is used.

This design ensures that:

- Long-running actions like movement can maintain continuous input across ticks without requiring the player to hold down keys.
- Single-tick actions (and the main game loop) can detect which keys are forced and respect them, avoiding conflicts.
- When an action completes, it can release its forced keys, restoring normal player control or allowing the next action to force different keys.

3.5.4 Action Sequencing and Execution Flow

The DSL interpreter produces an ordered sequence of `Action` objects, which are passed to the `ActionRunner` via an `ActionSequence`. The `ActionSequence` is a builder-like container that holds an immutable list of actions:

```
ActionSequence sequence = ActionSequence.start(action1)
    .then(action2)
    .then(action3);
runner.run(sequence);
```

When `runner.run(sequence)` is called, the `ActionRunner` replaces any existing queue and action with the new sequence's actions. On each subsequent game tick, the runner calls the current action's `tick()` method. When an action returns `DONE`, the runner polls the next action from the queue. If the queue becomes empty, the runner remains idle until a new sequence is submitted.

This design ensures:

- Actions are executed strictly sequentially, with each action completing before the next begins.
- Multi-tick actions automatically serialize with following actions; a movement command that takes 50 ticks will not overlap with subsequent inventory operations.
- The main game loop is never blocked; actions yield control every tick, allowing other game systems (rendering, entity updates, multiplayer synchronization) to proceed normally.

3.5.5 Cleanup and Error Recovery

The `stop` command and exception handling ensure that interrupted action sequences clean up their state properly. When an exception occurs during interpretation or action execution, or when the player manually stops execution:

- The `ActionRunner` is canceled, clearing the current action and queue.
- All forced input keys are released via `InputOverride.INSTANCE.releaseAll()`, restoring normal player control.
- The default keyboard input handler is restored via `MinecraftContext.restoreDefaultInput()`,

undoing any input overrides and ensuring player-controlled movement is possible.

– Any open inventory screens are left in their current state (to avoid abrupt UI changes that might confuse players), but subsequent player input is once again processed normally.

This recovery mechanism ensures that even in edge cases, the player regains full control of the game and the Minecraft client remains stable.

3.5.6 Design Rationale

The tick-based action model was chosen because it aligns naturally with Minecraft's game loop and provides several benefits:

– It provides Real-Time Interaction as the player can continue to observe and interact with the world while DSL actions execute, making the system feel responsive rather than blocking.

– The model is robust by relying on Minecraft's built-in input and physics systems rather than attempting to reimplement them, the system is robust to game updates, mod interactions, and edge cases in terrain or entity behavior.

– There is an established simplicity since each action class is stateful and self-contained, making it easy to reason about and test in isolation. Complex multi-tick behavior emerges from simple single-tick state machines.

– The system is extensible, since new actions can be added by implementing the `Action` interface and registering them in the standard library, without modifying the core runner or tick loop.

– There is also an aspect of observability, as players can enable debug logging to watch each action transition from `RUNNING` to `DONE`, aiding in learning and troubleshooting.

Conclusions

This project set out to design and partially implement CodeCraft, a domain-specific language for teaching programming to children aged 10–15 through the Minecraft game environment. The work completed across the two main chapters demonstrates that the goal is both technically feasible and educationally grounded.

The domain analysis established a clear motivation for the project. Existing tools either operate in abstract environments disconnected from children's interests, as is the case with Scratch, or impose technical barriers that are inappropriate for the target age group, as seen with ComputerCraft and raw Minecraft modding. Research reviewed during the analysis confirms that game-based, immediately visual programming environments measurably improve engagement, motivation, and the acquisition of computational thinking skills. Minecraft was identified as a particularly suitable host environment due to its existing adoption in formal education and its structural parallels to core programming concepts such as sequencing, iteration, and conditional logic.

The language design chapter translated these findings into concrete technical decisions. An imperative paradigm was selected for its low learning curve and alignment with sequential reasoning. Python-inspired syntax was adopted to minimize cognitive overhead, and snake_case identifier conventions were chosen based on empirical evidence showing lower visual effort for novice readers. The formal BNF grammar defines a coherent and complete language covering variables, two forms of iteration, conditionals, function calls, and comments. The lexer and parser implementations were described in sufficient detail to establish the correctness of the tokenization strategy, indentation handling, and recursive descent parsing approach.

Several limitations remain acknowledged. Installation still requires external assistance, the DSL is constrained to actions performable through the player character, and the current implementation covers lexing and parsing without yet including an interpreter or Minecraft integration layer. These constitute natural directions for future work, specifically the development of an AST evaluator, a Minecraft plugin bridge, and a beginner-friendly editor with inline error feedback.

Overall, CodeCraft demonstrates that a carefully scoped DSL can serve as a meaningful pedagogical bridge between block-based tools and general-purpose text languages, providing children with an entry point into real programming within a context they already value.

The full source code for the project, including the lexer, parser, and grammar specifications, is available at: [35].

Bibliography

[1] Tsai, M.-J.; Wang, C.-Y.; et al. Developing the Computer Programming Self-Efficacy Scale for Computer Literacy Education. *Journal of Educational Computing Research*, volume 56, no. 8, Jan. 2019: pp. 1345–1360, ISSN 0735-6331, 1541-4140, doi:10.1177/0735633117746747. Available from: https://journals.sagepub.com/doi/10.1177/0735633117746747

[2] Popat, S.; Starkey, L. Learning to code or coding to learn? A systematic review. *Computers & Education*, volume 128, Jan. 2019: pp. 365–376, ISSN 03601315, doi:10.1016/j.compedu.2018.10.005. Available from: https://linkinghub.elsevier.com/retrieve/pii/S0360131518302768

[3] Oscco Solorzano, R.; Torres Matos, L.; et al. Analysis of the Scratch program for computational thinking in schoolchildren in an educational institution in Peru: a network analysis. *Journal of Educational Technology Development and Exchange*, volume 18, no. 3, 2025: pp. 118–132, ISSN 19418035, doi:10.18785/jetde.1803.06. Available from: https://aquila.usm.edu/jetde/vol18/iss3/6/

[4] Kuz, A. Computational thinking: an analysis through structured programming using Scratch. *Revista de Ciencia y Tecnología*, , no. 39, May 2023: pp. 82–90, ISSN 18517587, 03298922, doi:10.36995/j.recyt.2023.39.010. Available from: https://www.fceqyn.unam.edu.ar/recyt/index.php/recyt/article/view/736

[5] Hlazova, V.; Monoharova, V.; et al. Integration of game elements and digital tools into modern methods of programming teaching. *E-learning teXnology*, volume 8, Dec. 2024: pp. 24–29, ISSN 2709-8400, doi:10.31865/2709-840082024316932. Available from: https://journals.uran.ua/texel/article/view/316932

[6] Medvedieva, M.; Zhmurko, O.; et al. Use of online game services in the study of programming languages. *Humanities science current issues*, volume 2, no. 36, 2021: pp. 248–255, ISSN 23084855, doi:10.24919/2308-4863/36-2-40. Available from: http://www.aphn-journal.in.ua/archive/36_2021/part_2/42.pdf

[7] Moshkova, N.; Aleksieieva, H.; et al. Gamification as One of the Trends of Modern Education. *Molod i rynok*, , no. 4/224, May 2024: pp. 82–87, ISSN 2617-0825, 2308-4634, doi:10.24919/2308-4634.2024.300147. Available from: http://mir.dspu.edu.ua/article/view/300147

[8] Faculty of Education in Jagodina, University of Kragujevac, Serbia; Milutinović, V. R.; et al. EXPLORING PRIMARY SCHOOL STUDENTS' MOTIVATION AND ENJOYMENT IN LEARNING PROGRAMMING THROUGH GAMIFICATION. In *STEM/STEAM/STREAM APPROACH IN THEORY AND PRACTICE OF CONTEMPORARY EDUCATION : Conference Proceedings*, University of Kragujevac, Faculty of Education in Jagodina, 2025, ISBN 978-86-7604-242-5, pp. 231–244,

doi:10.46793/STREAM25.231M. Available from: https://doi.ub.kg.ac.rs/doi/zbornici/10-46793-stream25-231m/?

[9] Domain-Specific Language | Computer Science | Research Starters | EBSCO Research. Available from: https://www.ebsco.com

[10] Fowler, M. *Domain-specific languages*. The Addison-Wesley signature series, Upper Saddle River, N.J: Addison-Wesley, 2011, ISBN 978-0-321-71294-3 978-0-13-210754-9.

[11] Connecting Blocks to Create Scripts in Scratch Programming Language - PiEmbSysTech - Embedded Systems & VLSI Lab. July 2024, section: Programming Languages. Available from: https://piembsystech.com/connecting-blocks-to-create-scripts-in-scratch-programming-language/

[12] Guide, C. Scratch vs. Other Coding Platforms: An In-depth Comparison. Oct. 2023. Available from: https://learncodingusa.com/scratch-vs-other-coding-platforms/

[13] Adler, F.; Fraser, G.; et al. Improving Readability of Scratch Programs with Search-based Refactoring.

[14] Licensing. Available from: https://education.minecraft.net/en-us/licensing

[15] Pros and Cons of Using Minecraft Education in the Classroom. Jan. 2025. Available from: https://www.schoolsthatlead.org/blog/2025/2/25/the-pros-and-cons-of-using-minecraft-education-in-the-classroom

[16] Sripan, T.; Manyam, K. Gamified Learning: Teaching Coding and Creative Thinking with Minecraft: Education Edition (M:EE) for Thai Students. *Journal of Education and Learning*, volume 14, no. 4, Mar. 2025: p. 270, ISSN 1927-5269, 1927-5250, doi:10.5539/jel.v14n4p270. Available from: https://ccsenet.org/journal/index.php/jel/article/view/0/51458

[17] Kazemitabaar, M.; Chyhir, V.; et al. Scaffolding Progress: How Structured Editors Shape Novice Errors When Transitioning from Blocks to Text. 2023, doi:10.48550/ARXIV.2302.05708, version Number: 1. Available from: https://arxiv.org/abs/2302.05708

[18] Md Shamsudin, N.; Dalim, S. F.; et al. Crafting STEM Lessons: A Systematic Literature Review on the Use of Minecraft in Education (2015-2024). *International Journal of Academic Research in Business and Social Sciences*, volume 14, no. 10, Oct. 2024: pp. Pages 2820–2827, ISSN 2222-6990, doi:10.6007/IJARBSS/v14-i10/23231. Available from: https://hrmars.com/journals/papers/IJARBSS/v14-i10/23231

[19] Nebel, S.; Schneider, S.; et al. Mining Learning and Crafting Scientific Experiments: A Literature Review on the Use of Minecraft in Education and Research. *Educational Technology & Society*, volume 19, Jan. 2016: pp. 355–366.

[20] Tablatin, C. L. S.; Casano, J. D. L.; et al. Using Minecraft to cultivate student interest in STEM. *Frontiers in Education*, volume 8, Mar. 2023: p. 1127984, ISSN 2504-284X, doi:10.3389/feduc.2023.

1127984. Available from: https://www.frontiersin.org/articles/10.3389/feduc.2023.1127984/full

[21] Short, D. Teaching Scientific Concepts using a Virtual World - Minecraft. *Teaching Science*, volume 58, Sept. 2012: pp. 55–58.

[22] Kotsopoulos, D.; Floyd, L.; et al. A Pedagogical Framework for Computational Thinking. *Digital Experiences in Mathematics Education*, volume 3, no. 2, Aug. 2017: pp. 154–171, ISSN 2199-3246, 2199-3254, doi:10.1007/s40751-017-0031-2. Available from: http://link.springer.com/10.1007/s40751-017-0031-2

[23] Ming, G. K.; Sirisena, A. Exploring the Impact of Minecraft-Based Projects on STEM and SDG Integration in Education: A Case Study. *International Journal of Research and Innovation in Social Science*, volume IX, no. IIIS, 2025: pp. 5242–5252, ISSN 24546186, doi:10.47772/IJRISS.2025.903SEDU0378. Available from: https://rsisinternational.org/journals/ijriss/articles/exploring-the-impact-of-minecraft-based-projects-on-stem-and-sdg-integration-in-edu

[24] Bower, M.; Wood, L.; et al. Improving the Computational Thinking Pedagogical Capabilities of School Teachers. *Australian Journal of Teacher Education*, volume 42, no. 3, Jan. 2017, ISSN 1835-517X, doi:10.14221/ajte.2017v42n3.4. Available from: https://ro.ecu.edu.au/ajte/vol42/iss3/4

[25] Noone, M.; Mooney, A. Visual and textual programming languages: a systematic review of the literature. *Journal of Computers in Education*, volume 5, no. 2, June 2018: pp. 149–174, ISSN 2197-9987, 2197-9995, doi:10.1007/s40692-018-0101-5. Available from: http://link.springer.com/10.1007/s40692-018-0101-5

[26] Saito, D.; Washizaki, H.; et al. Comparison of Text-Based and Visual-Based Programming Input Methods for First-Time Learners. *Journal of Information Technology Education: Research*, volume 16, Jan. 2017: pp. 209–226, doi:10.28945/3775.

[27] Neuwinger, M.; Riehle, D. A Systematic Review of Common Beginner Programming Mistakes in Data Engineering. Apr. 2025, doi:10.48550/arXiv.2504.16644, arXiv:2504.16644 [cs]. Available from: http://arxiv.org/abs/2504.16644

[28] Nadeem, A.; Rubaab, J.; et al. Enhancing Computational Thinking of Children with Dyslexia using Visual Programming Languages Model. *Journal of Computer Science & Computational Mathematics*, volume 11, no. 3, Sept. 2021: pp. 69–78, ISSN 22318879, doi:10.20967/jcscm.2021.03.007. Available from: https://www.jcscm.net/cms/?action=showpaper&id=2205238

[29] Weintrop, D.; Wilensky, U. Comparing Block-Based and Text-Based Programming in High School Computer Science Classrooms. *ACM Transactions on Computing Education*, volume 18, no. 1, Mar. 2018: pp. 1–25, ISSN 1946-6226, doi:10.1145/3089799. Available from: https://dl.acm.org/

doi/10.1145/3089799

[30] Akkaya, S.; Erkan, A. Examining Stakeholder Opinions on Coding Education in Primary Schools. *International Journal of Contemporary Educational Research*, volume 12, no. 1, Mar. 2025: pp. 1–25, ISSN 2148-3868, doi:10.52380/ijcer.2025.12.1.568. Available from: https://ijcer.net/index.php/pub/article/view/568

[31] Stakeholders Involved in BPC. July 2022. Available from: http://ecepalliance.org/resources/models/stakeholders-involved-bpc/

[32] Klimeková, E. Is Python an Appropriate Programming Language for Teaching Programming in Secondary Schools? *International Journal of Information and Communication Technologies in Education*, volume 4, May 2015, doi:10.1515/ijicte-2015-0005.

[33] Sharif, B.; Maletic, J. *An Eye Tracking Study on camelCase and under_score Identifier Styles*. Aug. 2010, doi:10.1109/ICPC.2010.41, journal Abbreviation: IEEE International Conference on Program Comprehension Pages: 205 Publication Title: IEEE International Conference on Program Comprehension.

[34] Ugwumba, N. *Programming Paradigm Learning Curves: An Empirical Study of Developer Skill Acquisition in Functional, Object-Oriented, and Imperative Programming*. Jan. 2026, doi:10.13140/RG.2.2.26177.01129.

[35] Boboc, C.; Brînza, V.; et al. CodeCraft: Learning Programming through Minecraft. https://github.com/Makday/CodeCraft, 2025. Available from: https://github.com/Makday/CodeCraft

[36] Fowler, M. DSL Guide. Aug. 2019. Available from: https://martinfowler.com/dsl.html

[37] Tomassetti, F. The complete guide to (external) Domain Specific Languages. Feb. 2017. Available from: https://tomassetti.me/domain-specific-languages/

[38] Guzdial, M. Software-Realized Scaffolding to Facilitate Programming for Science Learning. *Interactive Learning Environments*, volume 4, no. 1, Jan. 1994: pp. 001–044, ISSN 1049-4820, 1744-5191, doi:10.1080/1049482940040101. Available from: http://www.tandfonline.com/doi/abs/10.1080/1049482940040101

[39] Gu, P.; Wu, J.; et al. Scaffolding self-regulation in project-based programming learning through online collaborative diaries to promote computational thinking. *Education and Information Technologies*, volume 30, no. 12, Aug. 2025: pp. 16243–16267, ISSN 1360-2357, 1573-7608, doi:10.1007/s10639-025-13367-1. Available from: https://link.springer.com/10.1007/s10639-025-13367-1

[40] Ma, B.; Li, H.; et al. Scaffolding Metacognition in Programming Education: Understanding Student-AI Interactions and Design Implications. 2025, doi:10.48550/ARXIV.2511.04144, version Number:

1. Available from: https://arxiv.org/abs/2511.04144

[41] Belland, B. R.; Kim, C.; et al. A Framework for Designing Scaffolds That Improve Motivation and Cognition. *Educational Psychologist*, volume 48, no. 4, Oct. 2013: pp. 243–270, ISSN 0046-1520, 1532-6985, doi:10.1080/00461520.2013.838920. Available from: http://www.tandfonline.com/doi/abs/10.1080/00461520.2013.838920

[42] Repenning, A.; Grabowski, S. Scaffolding Creative Programming Projects. In *Proceedings of the 19th WiPSCE Conference on Primary and Secondary Computing Education Research*, Munich Germany: ACM, Sept. 2024, ISBN 979-8-4007-1005-6, pp. 1–6, doi:10.1145/3677619.3677634. Available from: https://dl.acm.org/doi/10.1145/3677619.3677634

[43] Qian, Y.; Lehman, J. Students' Misconceptions and Other Difficulties in Introductory Programming: A Literature Review. *ACM Transactions on Computing Education*, volume 18, no. 1, Mar. 2018: pp. 1–24, ISSN 1946-6226, doi:10.1145/3077618. Available from: https://dl.acm.org/doi/10.1145/3077618

[44] Microsoft Computer Science Curriculum Whitepaper. Available from: https://www.microsoft.com/education

[45] Vol.14, No.4 | Journal of Education and Learning | CCSE. Available from: https://ccsenet.org/journal/index.php/jel/issue/view/0/3200

[46] Stalla, A. DSL for Modding Minecraft. Feb. 2024. Available from: https://tomassetti.me/dsl-for-minecraft/

[47] Minescript – Python scripting for Minecraft. Available from: https://minescript.net/

[48] SpigotDSL. Available from: https://www.spigotmc.org/resources/spigotdsl.79383/

[49] Enea, A. *Closing the Block-to-Text Gap: A Domain-Specific JavaScript Editor for Early Computational Thinking*. Oct. 2025, doi:10.48550/arXiv.2512.00012.

[50] Hermans, F. Hedy: A Gradual Language for Programming Education. In *Proceedings of the 2020 ACM Conference on International Computing Education Research*, ICER '20, New York, NY, USA: Association for Computing Machinery, 2020, ISBN 978-1-4503-7092-9, pp. 259–270, doi:10.1145/3372782.3406262. Available from: https://dl.acm.org/doi/10.1145/3372782.3406262

[51] Stefik, A.; Siebert, S. An Empirical Investigation into Programming Language Syntax. *ACM Transactions on Computing Education*, volume 13, no. 4, Nov. 2013: pp. 1–40, ISSN 1946-6226, doi:10.1145/2534973. Available from: https://dl.acm.org/doi/10.1145/2534973

[52] Volina, N.; Hlavacs, H. *Enhancing Programming learnability for Children through Video Games*, volume 19. Sept. 2025, doi:10.34190/ecgbl.19.2.3981, journal Abbreviation: European Conference on Games Based Learning Publication Title: European Conference on Games Based Learning.

A BNF Grammar

```
<program>           ::= <statement_list>

<statement_list>    ::= <statement>
                      | <statement> <statement_list>

<statement>         ::= <assignment> NEWLINE
                      | <for_count>
                      | <for_range>
                      | <while_loop>
                      | <if_stmt>
                      | "stop" NEWLINE
                      | <function_call> NEWLINE
                      | <comment>

<assignment>        ::= <identifier> <optional_space> "=" <optional_space>
    <expression>

<for_count>         ::= "for" <space> <expression> <space> "times" NEWLINE
    <block>
<for_range>         ::= "for" <space> <identifier> <space> "from" <space>
   <expression> <space> "to" <space> <expression> NEWLINE <block>
<while_loop>        ::= "while" <space> <condition> NEWLINE <block>
<if_stmt>           ::= "if" <space> <condition> NEWLINE <block>
                      | "if" <space> <condition> NEWLINE <block> "else"
                         NEWLINE <block>

<block>             ::= INDENT <statement_list> DEDENT

-- Function call
<function_call>     ::= <identifier> "()"
                      | <identifier> "(" <argument_list> ")"
```

```
<argument_list>     ::= <expression>
                      | <expression> <optional_space> "," <optional_space>
                          <argument_list>

<condition>         ::= <expression> <optional_space> <comparison_op> <
   optional_space> <expression>
                      | "not" <space> <condition>
                      | <function_call>
                      | <identifier>

<comparison_op>     ::= "==" | "!=" | "<" | ">" | "<=" | ">="

<expression>        ::= <term>
                      | <expression> <optional_space> "+" <optional_space>
                          <term>
                      | <expression> <optional_space> "-" <optional_space>
                          <term>

<term>              ::= <factor>
                      | <term> <optional_space> "*" <optional_space> <
                         factor>
                      | <term> <optional_space> "/" <optional_space> <
                         factor>

<factor>            ::= <literal>
                      | <identifier>
                      | <special_object>
                      | <function_call>
                      | "(" <optional_space> <expression> <optional_space>
                          ")"
                      | "-" <factor>

<special_object>    ::= <identifier>"."<identifier>

<literal>           ::= <number>
                      | <string>
```

```
                    | <boolean>

<boolean>           ::= "true" | "false"
<number>            ::= <digit> | <digit> <number>
<digit>             ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" |
   "8" | "9"
<string>            ::= "\"" <characters> "\""
<characters>        ::= ε | <character> <characters>
<character>         ::= <letter> | <special_char>

<identifier>        ::= <letter> | <letter> <id_tail>
<id_tail>           ::= <id_char> | <id_char> <id_tail>
<id_char>           ::= <letter> | <digit> | "_"

<letter>            ::= <lower> | <upper>

<lower>             ::= "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h" | "i
   " | "j" | "k" | "l" | "m"
                    | "n" | "o" | "p" | "q" | "r" | "s" | "t" | "u" | "v
                       " | "w" | "x" | "y" | "z"

<upper>             ::= "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H" | "I
   " | "J" | "K" | "L" | "M"
                    | "N" | "O" | "P" | "Q" | "R" | "S" | "T" | "U" | "V
                       " | "W" | "X" | "Y" | "Z"

<special_char>      ::= "!" | "\"" | "#" | "$" | "%" | "&" | "'" | "(" |
   ")"
                    | "*" | "+" | "," | "-" | "." | "/" | ":" | ";" |
                       "<"
                    | "=" | ">" | "?" | "@" | "[" | "\\" | "]" | "^" | "
                       _"
                    | "`" | "{" | "|" | "}" | "~" | <space>

<space>             ::= " "
<optional_space>    ::= <space> | ε
```

```
<comment>            ::= "--" <characters> NEWLINE
```

B Team Contract

This appendix contains the signed team contract agreed upon at the start of the project.

Team Members and Roles

Member 1: Bruma Cristian - Project Manager

Responsibilities:

- Coordinate team meetings and maintain meeting notes
- Oversee project timeline and milestone completion
- Lead lexer and parser implementation
- Develop abstract syntax tree (AST) structure
- Coordinate integration between frontend and backend components
- Monitor overall project progress and report blockers

Weekly Time Commitment: 8-10 hours

Member 2: Bîtca Gabriela - Language Designer & Documentation Lead

Responsibilities:

- Design DSL syntax and semantics
- Create and maintain formal grammar (BNF notation)
- Define Minecraft-specific language constructs (blocks, entities, world manipulation)
- Write Chapters 1 and 2 of the final report
- Maintain Zotero bibliography (minimum 20 references)
- Design example programs demonstrating DSL capabilities

Weekly Time Commitment: 8-10 hours

Member 3: Brînză Vasile - Backend Developer & Code Generator

Responsibilities:

- Implement semantic analyzer and symbol table
- Develop code generation module (transpiler to Minecraft commands/Java)
- Create runtime environment or interpreter
- Write Chapter 3 of the final report (implementation details)
- Ensure proper error handling and validation
- Optimize generated code

Weekly Time Commitment: 8-10 hours

Member 4: Strulea Teodor - Testing Lead & Quality Assurance

Responsibilities:

- Develop comprehensive test suite for DSL
- Create sample Minecraft programs using the DSL
- Test language features and edge cases
- Document bugs and coordinate fixes
- Prepare technical demonstrations for mid-term evaluations
- Write testing and validation sections of report

Weekly Time Commitment: 8-10 hours

Member 5: Boboc Chiril - Research Lead & Scientific Paper Author

Responsibilities:

- Conduct literature review on DSLs and Minecraft modding
- Write scientific paper abstract and full paper
- Prepare presentations for mid-term and final evaluations
- Compile and edit final report from all members' contributions
- Create PBL Problem Map presentation
- Format final deliverables according to university guidelines

Weekly Time Commitment: 8-10 hours

Team Expectations and Ground Rules

Communication

1. **Response Time:** All team members will respond to messages within 24 hours
2. **Primary Channels:**
    - Weekly team meetings: [Day and Time]
    - Chat platform: [Discord/WhatsApp/Telegram]
    - Code repository: [GitHub/GitLab]
3. **Meeting Attendance:** Mandatory attendance at weekly meetings (online or in-person)
4. **Mentor Meetings:** All members present unless excused with 24-hour notice

Work Standards

1. **Code Quality:** All code must be documented and follow agreed-upon style guides
2. **Deliverable Deadlines:** Individual tasks due 48 hours before team deadlines
3. **Peer Review:** All major contributions require review by at least one other member
4. **Documentation:** Work must be documented as completed, not after the fact
5. **Version Control:** Commit code regularly with clear, descriptive messages

Collaboration

1. **Knowledge Sharing:** Members will maintain shared documentation of their work
2. **Cross-Training:** Each member should understand basics of all project components
3. **Flexible Roles:** Members assist others when ahead on personal tasks
4. **Constructive Feedback:** All criticism must be specific, actionable, and respectful
5. **Inclusive Decision-Making:** Major decisions require majority agreement

---

Weekly Workflow

Before Weekly Meeting

- Complete assigned tasks for the week
- Update task tracker/project board
- Prepare questions or blockers for discussion
- Review others' contributions if assigned

During Weekly Meeting

- Report on completed tasks and progress
- Discuss challenges and blockers
- Plan next week's tasks
- One member takes rotating meeting notes
- Identify items needing mentor guidance

After Weekly Meeting

- Update individual project diary
- Begin assigned tasks for upcoming week
- Follow up on action items within 48 hours

Deliverables Timeline and Responsibilities

Week	Deliverable	Primary Responsible
1	Domain selection, problem identification	Member 2
2	PBL Problem Map, domain analysis	Member 5
3	Literature analysis (20+ refs)	Member 5
4	Project plan	Member 1, 5
5	Formal grammar (BNF)	Member 2
6	Lexer/parser, AST	Member 1
7	Scientific paper abstract	Member 5
8-9	Mid-term I presentation	Member 5
10-11	Frontend implementation complete	Member 1
12	Scientific paper draft	Member 5
13	Technical demo (Mid-term II)	Member 4
14	Final report draft	Member 5
15	Final presentation	All

Breach of Contract and Consequences

Minor Violations (First Offense)

Examples: Late response, minor missed deadline, incomplete meeting notes

Consequences:

- Written warning documented in peer review
- Make up work by completing extra documentation or testing
- Buy team snacks/drinks for next meeting

Moderate Violations (Pattern or Second Offense)

Examples: Missed meeting without notice, repeatedly late deliverables, poor quality work

Consequences:

- Formal meeting with team to discuss improvement plan
- Additional compensatory work (e.g., extra test cases, documentation sections)
- Team treats (pizza/coffee for the team)
- Mentor notification in peer review

Severe Violations

Examples:

- 4 unexcused absences by mid-term
- Deliberate sabotage or non-contribution
- Plagiarism or academic dishonesty
- Complete absence from project for 2+ consecutive weeks

Consequences:

- Immediate meeting with mentor
- Meeting with mentor and head of study program
- Possible removal from team (per Free-Riding Protocol)
- Individual project requirement with same deadlines
- Grade penalty as determined by mentor

Positive Recognition

Team members who consistently exceed expectations will receive:

- Positive mention in peer reviews
- Recognition in final presentation
- Potential grade boost recommendation to mentor
- Team acknowledgment and appreciation

Conflict Resolution Process

1. **Direct Discussion (First Step):** Members address issues directly and respectfully
2. **Team Mediation (If Unresolved):** Bring to full team meeting for discussion
3. **Mentor Involvement (Escalation):** Request mentor guidance if internal resolution fails
4. **Formal Action (Last Resort):** Follow university protocols for severe cases

---

Vow of Commitment

I, member of this PBL group, commit to:

- Attending all scheduled meetings or providing 24-hour notice for absences
- Completing my assigned tasks on time and to high standards
- Communicating proactively about challenges or delays
- Respecting my teammates' time and contributions
- Maintaining professionalism and constructive collaboration
- Taking responsibility for my role in the project's success
- Providing honest peer reviews and accepting feedback graciously
- Supporting team members when they face difficulties
- Upholding academic integrity in all project work
- Attending meetings caffeinated enough to distinguish between Java (the programming language) and Java (the coffee), though both are equally essential to this project.
- That if the project crashes and burns, we all go down with the ship like true captains—but hopefully we'll sail smoothly to a grade 10 instead.
- Read the documentation before asking teammates for help, unlike the usual workflow of Ctrl+C, Ctrl+V, and pray.
- Contributing equally to the team's workload (approximately 8-10 hours/week)

Additional Notes

This contract may be amended with unanimous agreement of all team members. Any amendments must be documented, dated, and signed by all members. This contract will be reviewed at the mid-term evaluation points and adjusted if necessary.

Next Review Date: [Date of Mid-term I]

Team Agreement: We agree to revisit this contract if team dynamics change or if the project scope is significantly adjusted.

UNIVERSITATEA TEHNICĂ A MOLDOVEI

Team Contract

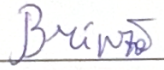
CodeCraft

Semester: 4
Project Theme: Design and Implementation of a DSL for Minecraft Automation and Scripting
Project Period: February - June 2026
ECTS: 8
Project Group Number: 8

Supervisory Team:
Supervisors: Cojuhari Irina
Mentors: Crucerescu Vladislav

[Boboc Chiril]

[Brînză Vasile]



[Bruma Cristian]

[Bîtca Gabriela]

[Strulea Teodor]

By signing this document, each member of the group confirms that they are participating on equal terms in the creation and development of the project and agrees to the terms and conditions of this contract.

Chișinău, 2025

C PBL Problem Map

TYPE OF QUESTION	EXAMPLES OF QUESTIONS	SHORT ANSWER
a)	Present the problem situation in 5–7 sentences.	Children today face unprecedented challenges with attention span and sustained focus, making traditional programming education—which often involves lengthy tutorials, abstract concepts, and delayed gratification—increasingly ineffective. Gamification has emerged as a powerful educational strategy that leverages game mechanics, instant feedback, and intrinsic motivation to maintain engagement and facilitate learning. Minecraft represents an ideal platform for gamified programming education, as children are already deeply invested in the game and motivated to enhance their gameplay experience. However, existing Minecraft programming solutions are too complex for young learners and fail to incorporate gamified learning principles that could sustain their attention. There is a critical need for a domain-specific language that transforms programming education into an engaging, game-like experience within Minecraft, where children see immediate results from their code through scripted actions executed in the game.

b)	Formulate the problem – 1 statement.	Children's limited attention spans prevent them from learning programming through traditional methods, which often lack immediate visual feedback and engaging concepts.
Who...?	1. Who has a problem? 2. Who is involved in the problem situation? 3. Who suffers from the problem? 4. Who should take part in solving the problem?	1. Children. 2. Children, parents, professionals of the educational domain, authorities concerned with education. 3. Children. 4. Authorities concerned with education, DSL designers and developers, teachers.
What...? Which...?	5. What happened? 6. What is the problem? 7. What led you to identify the problem? 8. What are the causes that affect those involved? 9. What are the needs of those targeted? 10. What resources are necessary to solve the problem?	5. Short-form media has become a main source of information. 6. Incompatibility between traditional programming education and the attention patterns of modern children. 7. Research showing declining attention spans in children. 8. Fast-paced media consumption; limited availability of age-appropriate engaging programming tools. 9. Immediate visual feedback, game-based learning, progressive difficulty, creative freedom. 10. DSL developers, educational consultants, UI/UX designers, Minecraft experts, child testers, teacher advisors.

Where…?	11. Where did the problem arise?	11. At the intersection of children's educational environments (homes, schools, after-school programs) and their recreational gaming spaces.
When…?	12. When did the problem arise? 13. When was the problem identified? 14. When is it planned to be resolved?	12. Progressively over the past decade (2015–2025), as exposure to fast-paced digital media increased. 13. During the initial stages of the PBL project (2025–2026 academic year). 14. By the end of the current semester.
Why…?	15. Why is there a need to solve the problem?	15. Programming and computational thinking are essential 21st-century skills. Without intervention, an entire generation may miss foundational digital literacy skills necessary for future academic and career success.
How…?	16. How can the problem be solved? 17. How will the elimination of the problem be determined?	16. Gamification of the learning process. 17. By the number of users of the solution and the number of educational systems that have adopted it.
How much…?	18. How big is the problem? 19. How many people are affected? 20. How long will it take to resolve? 21. How many parties need to be involved?	18. Global — affects generations of children with excessive exposure to digital technologies. 19. Estimated at over 140 million children. 20. Implementation could potentially be completed in 6 months. 21. Minimum 3 parties.

Other (optional)	22. How do you see solving this problem? 23. Conclusions and reflections.	22. The product should contain simplified programming syntax, educator resources, and use-case tutorials suited for children. Given Minecraft's popularity, success potential is high. 23. The Minecraft Bot Maker DSL addresses a genuine and growing educational challenge; if done right, it could become a fundamental learning tool.

D Project Plan

This appendix presents the project schedule, deliverables, and team responsibilities across the full 15-week development cycle. The project is structured into four sequential phases, each corresponding to a distinct stage of the software engineering process: problem identification and research, language planning and grammar design, core development and milestone evaluation, and final delivery.

Development Phases

Phase 1 — Problem & Research (Weeks 1–3) Covers the initial definition of the problem domain, the construction of the PBL Problem Map, and a systematic literature review comprising a minimum of twenty references.

Phase 2 — Planning & Grammar (Weeks 4–6) Encompasses the preparation of the project plan, the formal specification of the language grammar in Backus–Naur Form, and the implementation of the lexer, parser, and Abstract Syntax Tree.

Phase 3 — Development & Milestones (Weeks 7–13) Covers the writing of the scientific paper abstract and draft, the Mid-term I and Mid-term II presentations, and the completion of the frontend implementation.

Phase 4 — Final Delivery (Weeks 14–15) Comprises the submission of the final report draft and the delivery of the final project presentation before the full team and evaluators.

Deliverables Timeline and Responsibilities

The table below enumerates all thirteen project deliverables, their associated week, and the team member responsible for each.

Week	Deliverable	Responsible
Phase 1 — Problem & Research		
W1	Domain selection and problem identification	Member 2
W2	PBL Problem Map and domain analysis	Member 5
W3	Literature analysis (minimum 20 references)	Member 5
Phase 2 — Planning & Grammar		
W4	Project plan	Members 1, 5
W5	Formal grammar specification (BNF)	Member 2
W6	Lexer, parser, and Abstract Syntax Tree	Member 1
Phase 3 — Development & Milestones		
W7	Scientific paper abstract	Member 5
W8–9	Mid-term I presentation	Member 5
W10–11	Frontend implementation complete	Member 1
W12	Scientific paper draft	Member 5
W13	Technical demonstration (Mid-term II)	Member 4
Phase 4 — Final Delivery		
W14	Final report draft	Member 5
W15	Final presentation	All Members

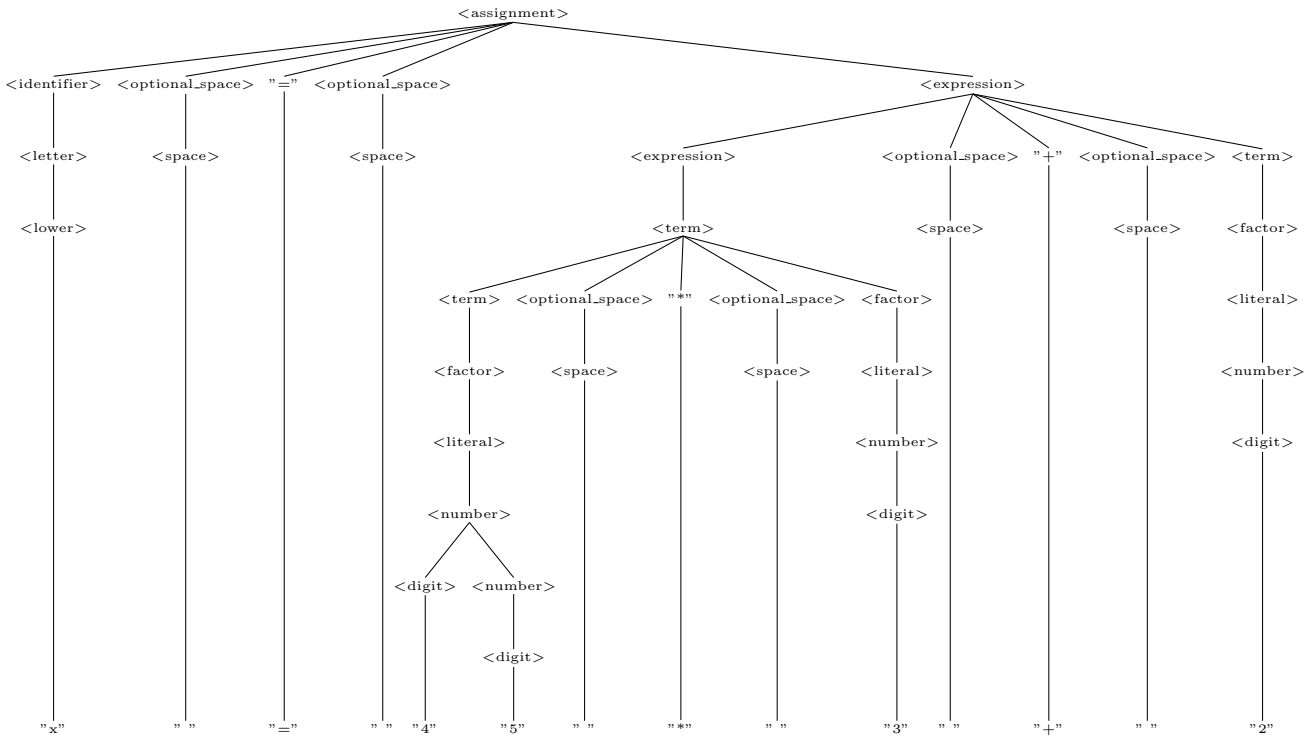
Total: 15 weeks, 13 deliverables, 5 team members.

E Derivation Trees and Abstract Syntax Trees

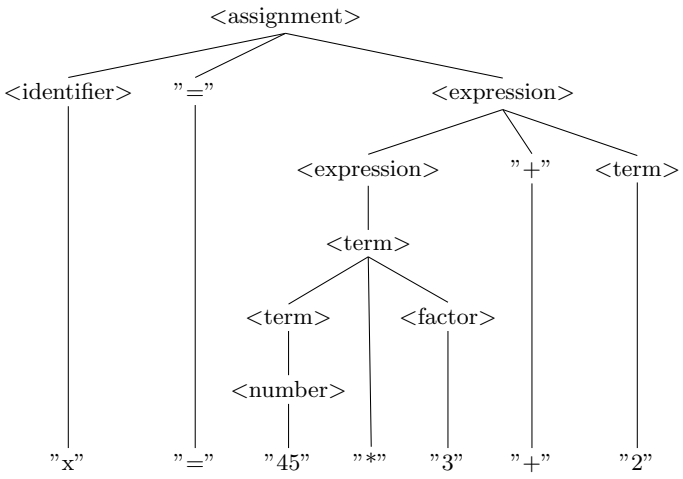
Derivation trees of CodeCraft, a Domain-Specific Scripting Language for Minecraft

1 Variable Asssignment

Derivation Tree



Simplified Abstract Syntax Tree



The derivation tree will produce the result:

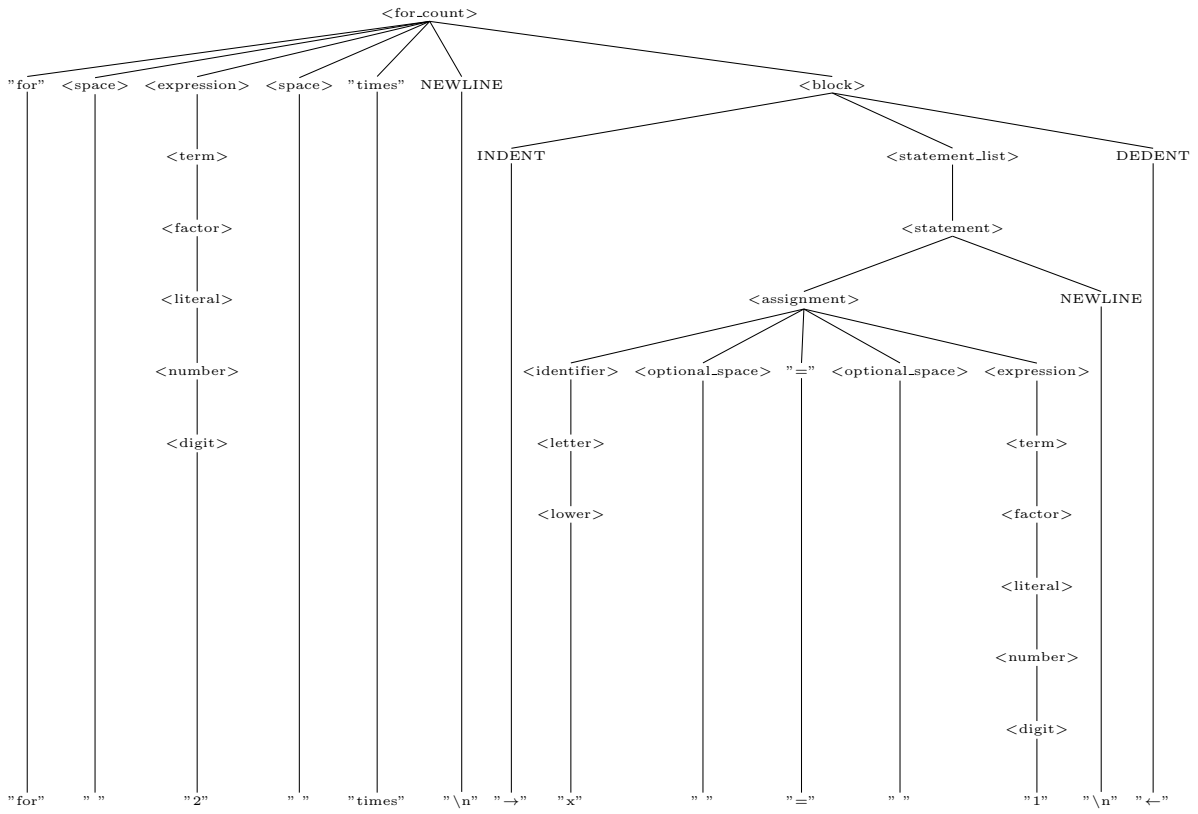
x = 45 * 3 + 2

The tree shows that the operator precedence is handled correctly, along with the possibility to nest expressions.

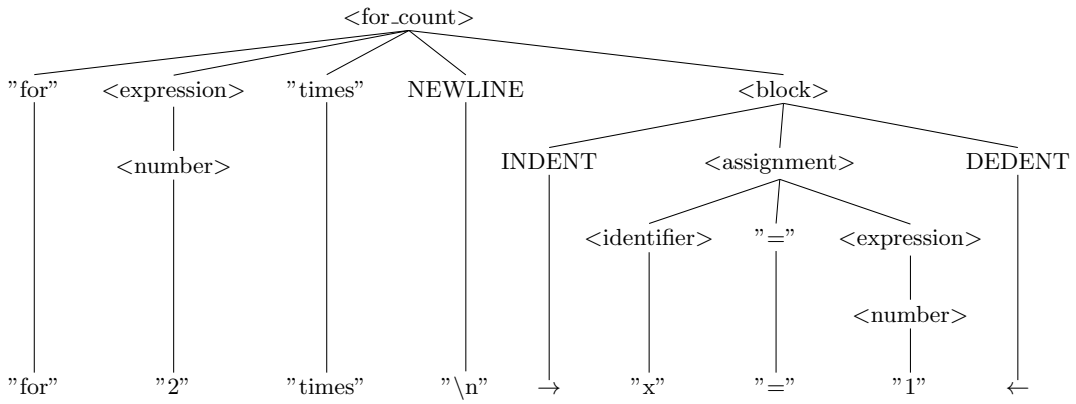
2 Control Structures

2.1 For Count

Derivation Tree



Simplified Abstract Syntax Tree



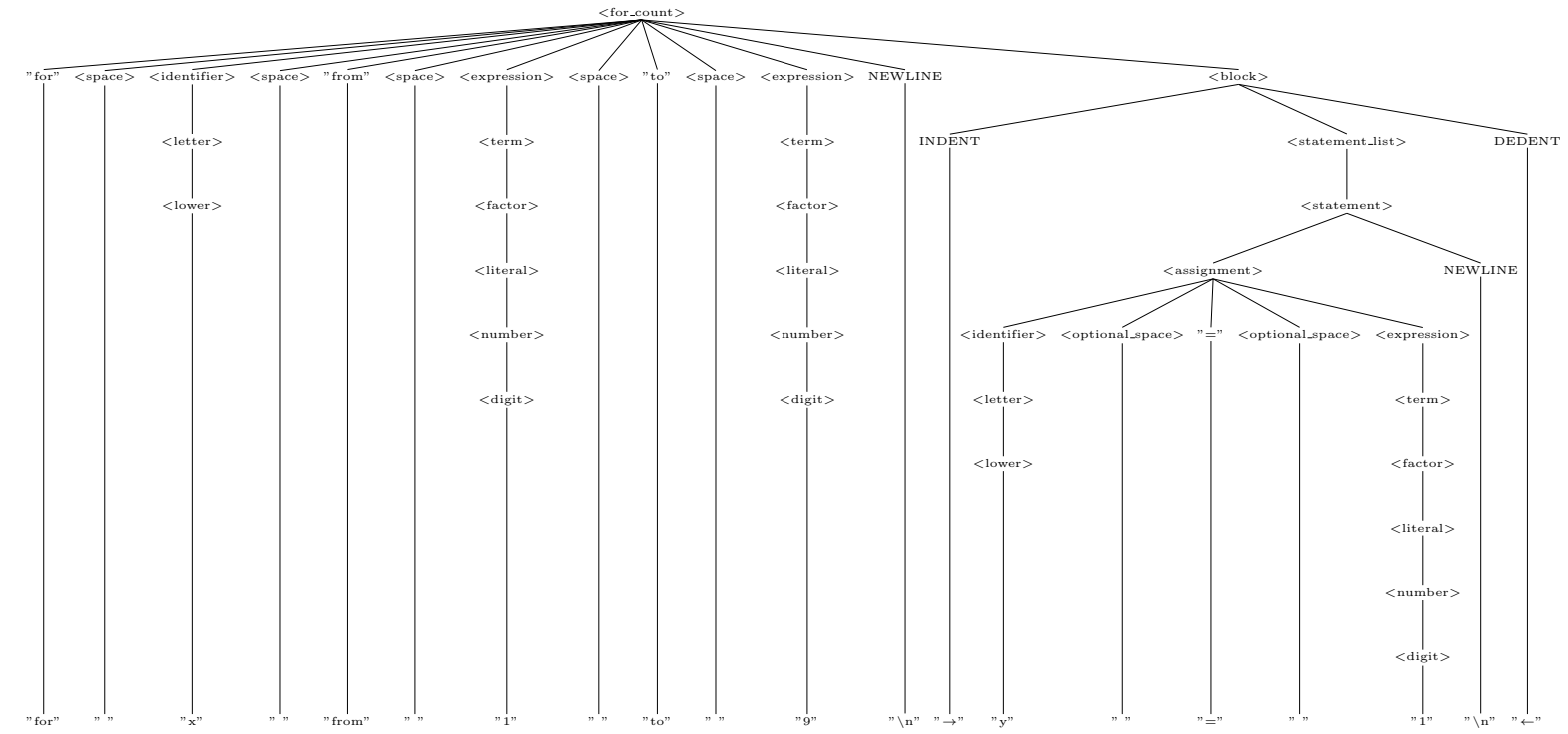
The derivation tree will produce the result:

```
for 2 times
    x = 1
```

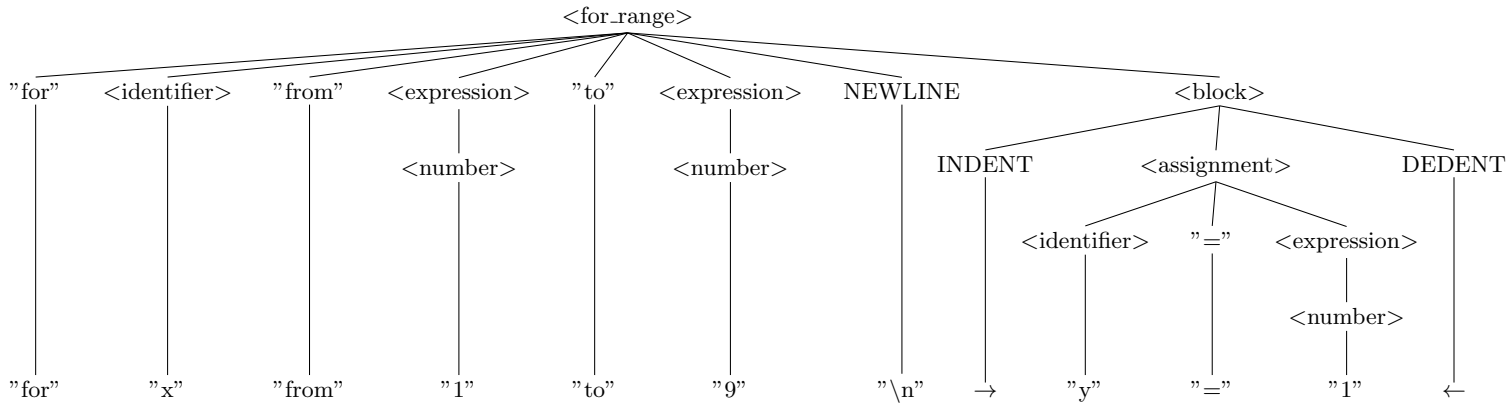
The tree illustrates that the block scopes are handled through forced indentations and dedentations, a practice similar to *Python*.

2.2 For Range

Derivation Tree



Simplified Abstract Syntax Tree



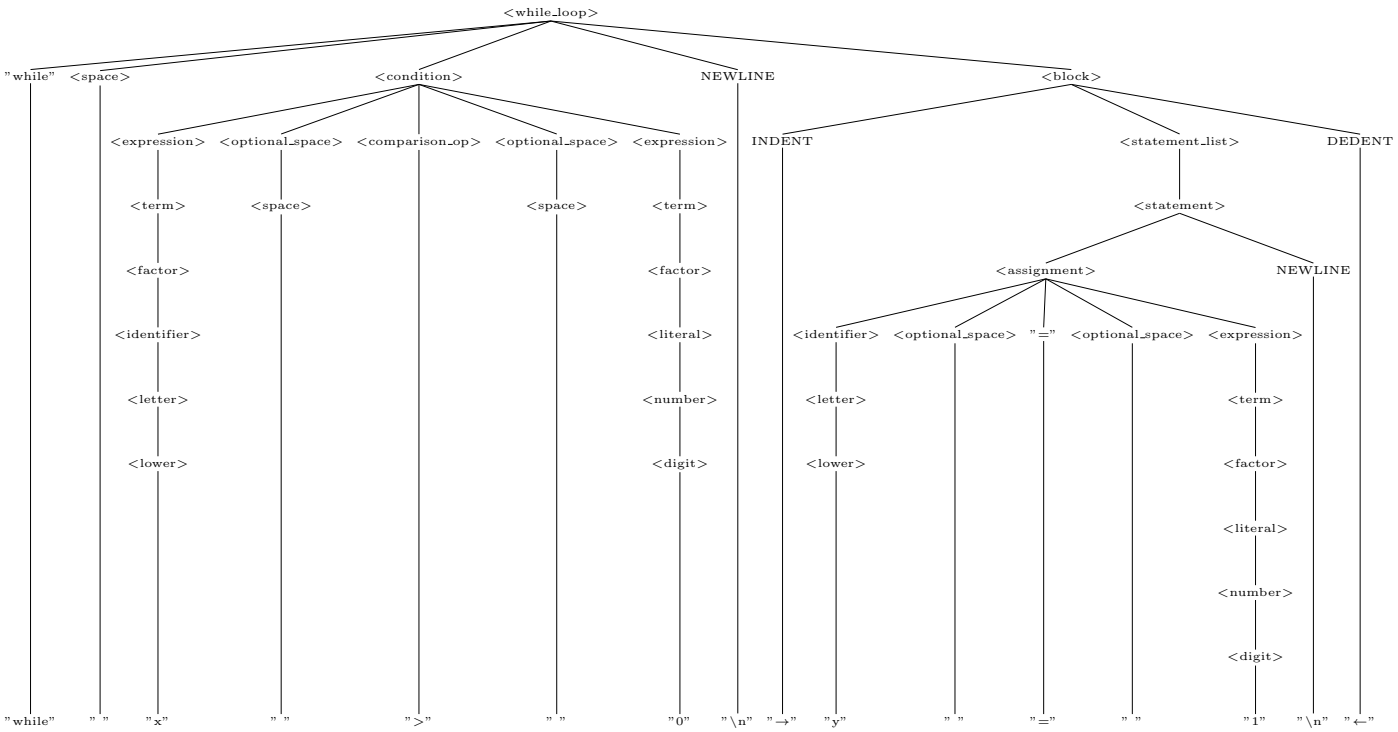
The derivation tree will produce the result:

```
for x from 1 to 9
    y = 1
```

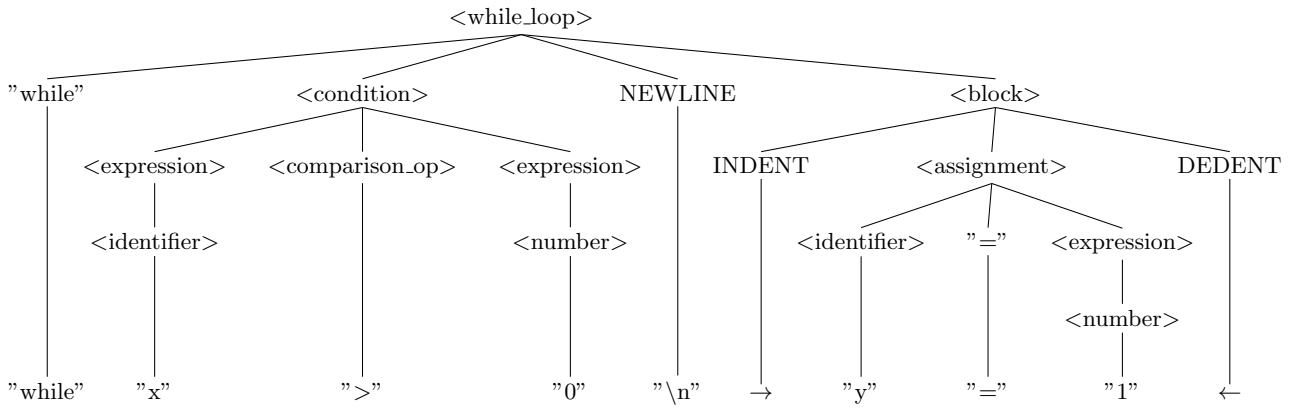
The purpose of the *For Range* is to give the user the option to iterate via a variable. In this case, during the iterations, x will take values from *1* to *9*, in this exact order. If the range is set in a non-descending manner like *from 6 to 1*, the compiler will identify that and adjust the iteration accordingly, meaning that during the iterations, x will take values from *6* to *1*, in this exact order.

2.3 While Loop

Derivation Tree



Simplified Abstract Syntax Tree

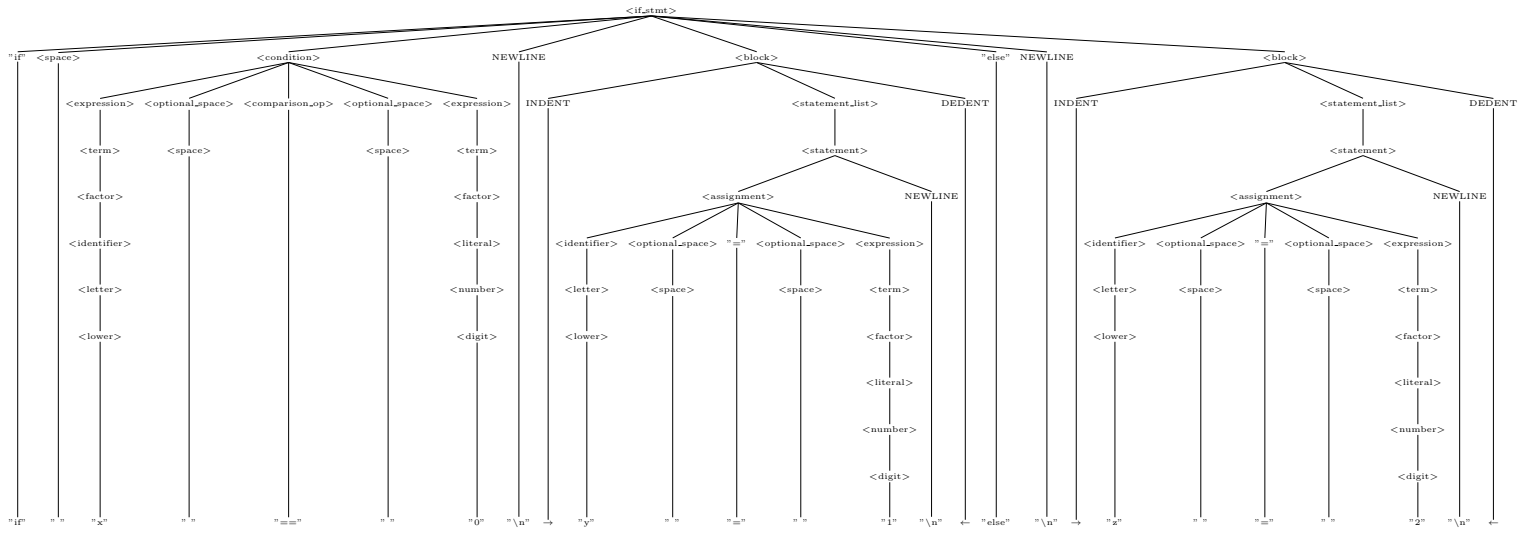


The derivation tree will produce the result:

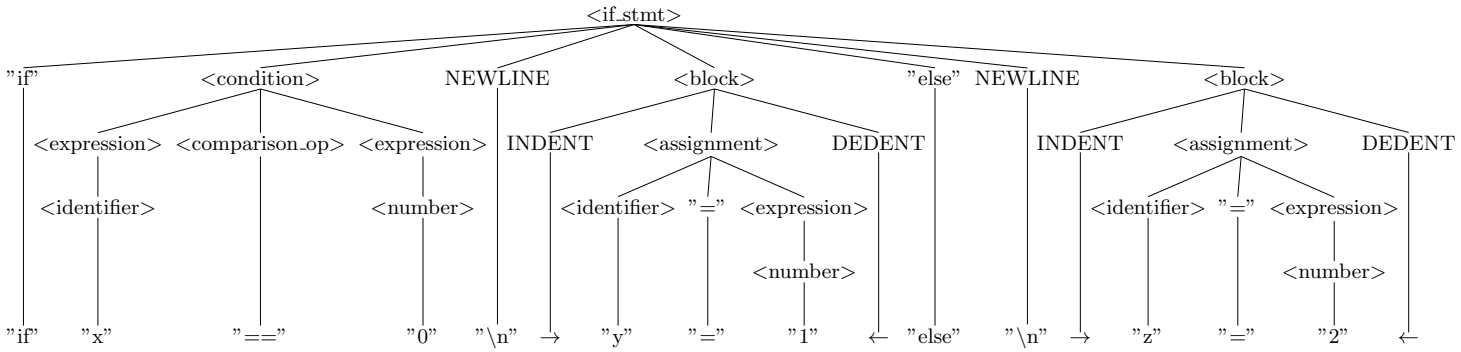
```
while x > 0
    y = 1
```

2.4 If-Else Statement

Derivation Tree



Simplified Abstract Syntax Tree



The derivation tree will produce the result:

```
if x == 0
    y = 1
else
    z = 2
```